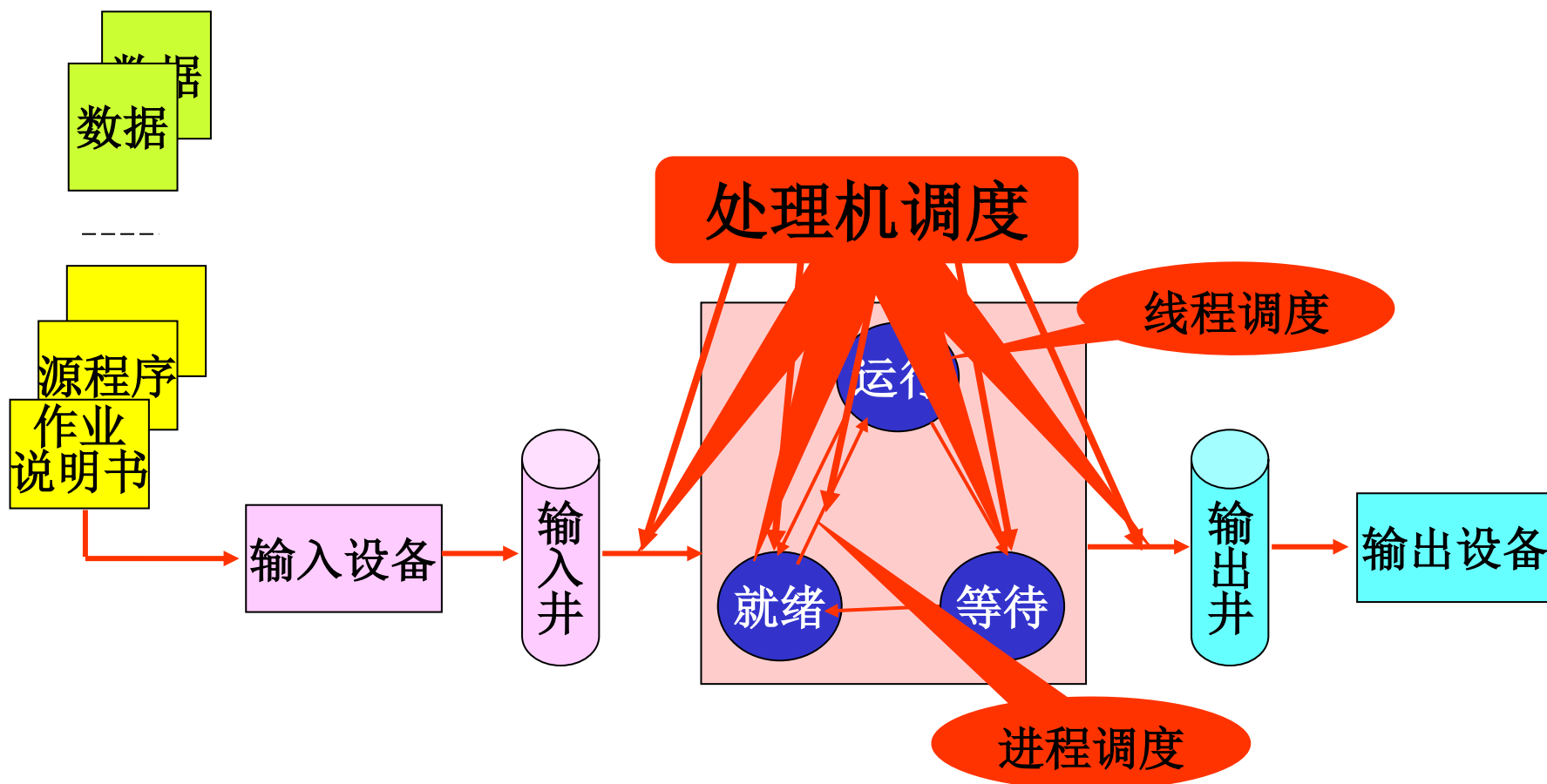
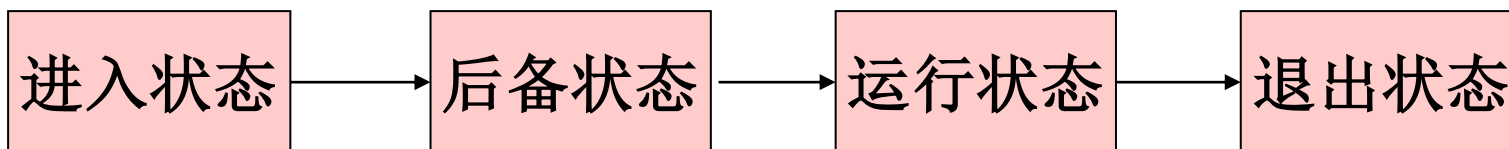




Review-还差什么



- 作业管理和用户接口
 - ✓ 作业的概念、作业说明书、JCB、作业状态
 - ✓ 如何选择作业执行？
- 进程管理
 - ✓ 进程概念、进程描述（PCB）、进程状态
 - ✓ 线程：概念、实现方式
 - ✓ 进程控制、原语
- 进程同步与互斥
 - ✓ 互斥锁（软件、硬件）、信号量、管程
- 进程间通信IPC：低级通信、高级通信
 - ✓ 消息机制、共享存储器、管道、套接字、信号
- 死锁：一种互相等待的状态
 - ✓ 概念、原因、条件；解决办法：预防、避免、检测与恢复
 - ✓ 死锁避免的银行家算法、死锁检测的死锁定理和检测算法
- 如何选择一个进程/线程执行，即进程/线程调度？





第七章

处理机调度





第七章 处理机调度

- **需求：**对CPU资源进行合理的分配使用，支持多种**调度策略**（普通、实时）、**多处理机调度**、**多层级**（作业、进程、线程），又要高的资源利用率和高效率
- **现状：**任务多处理器少（“僧多粥少”），
- **目标：**处理机**利用率高**，**公平**，**不饿死**，**响应快**
- **解决的问题**
 - ✓ **WHAT：**按什么原则分配CPU — 进程调度算法
 - ✓ **WHEN：**何时分配CPU — 进程调度的时机
 - ✓ **HOW：**如何分配CPU — CPU调度过程（包括进程的上下文切换）



第七章 处理机调度

内容

1. 分级调度
2. 作业调度概述
3. 进程调度概述
4. 调度算法
5. 调度算法运行举例
6. 实时调度
7. 多处理机调度
8. Linux(openEuler)的进程调度简介

本章目标:

1. 理解处理机调度的层次，能够**从不同的角度**认识处理机调度；
2. 理解作业/进程调度的评价指标，能够利用这些指标评价具体的调度算法；
3. **掌握作业/进程调度的几种基本算法，能够根据具体的算法给出调度序列，并进行定量评价**
4. 了解多处理机调度面临的问题和挑战；
5. 了解Linux（openEuler）的调度策略、算法等



7.1 分级调度

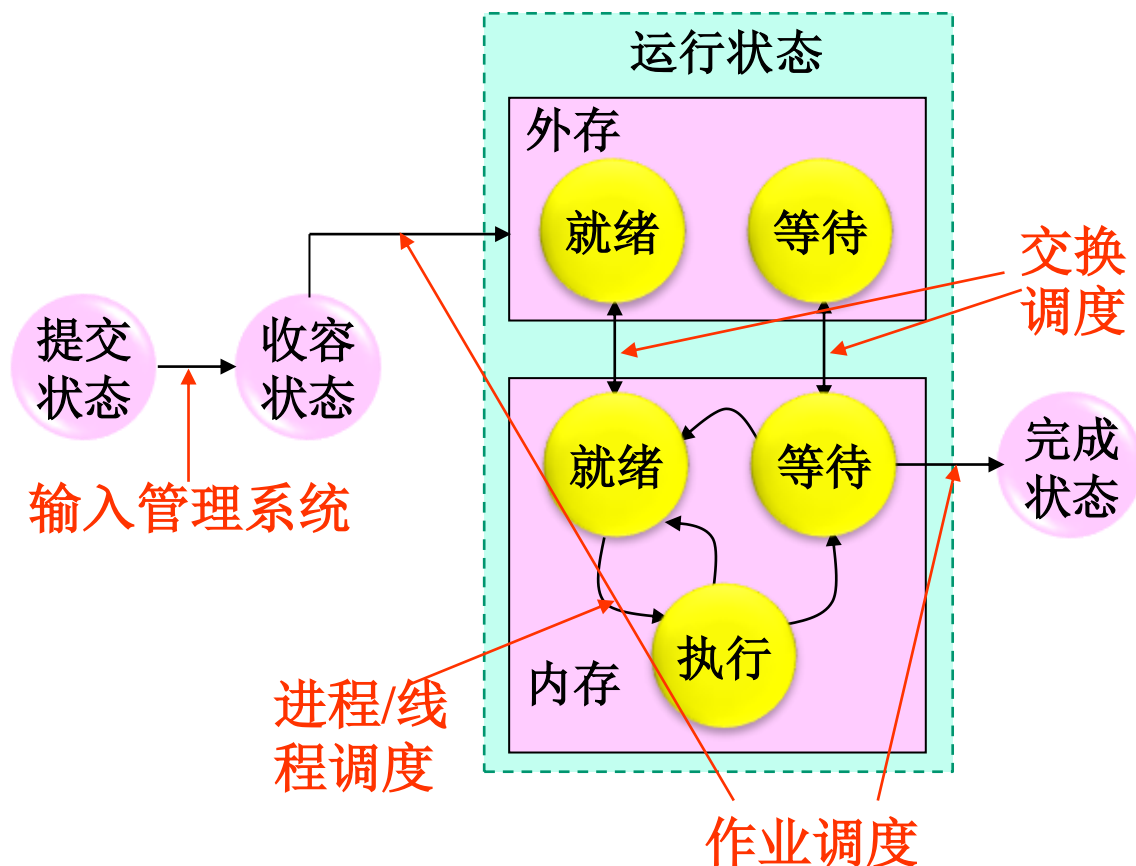
内容:

1. 作业的状态及其转换
2. 调度的层次
3. 调度时间周期
4. OS类型
5. 再认识作业与进程的关系



7.1 分级调度

7.1.1 作业的状态及其转换



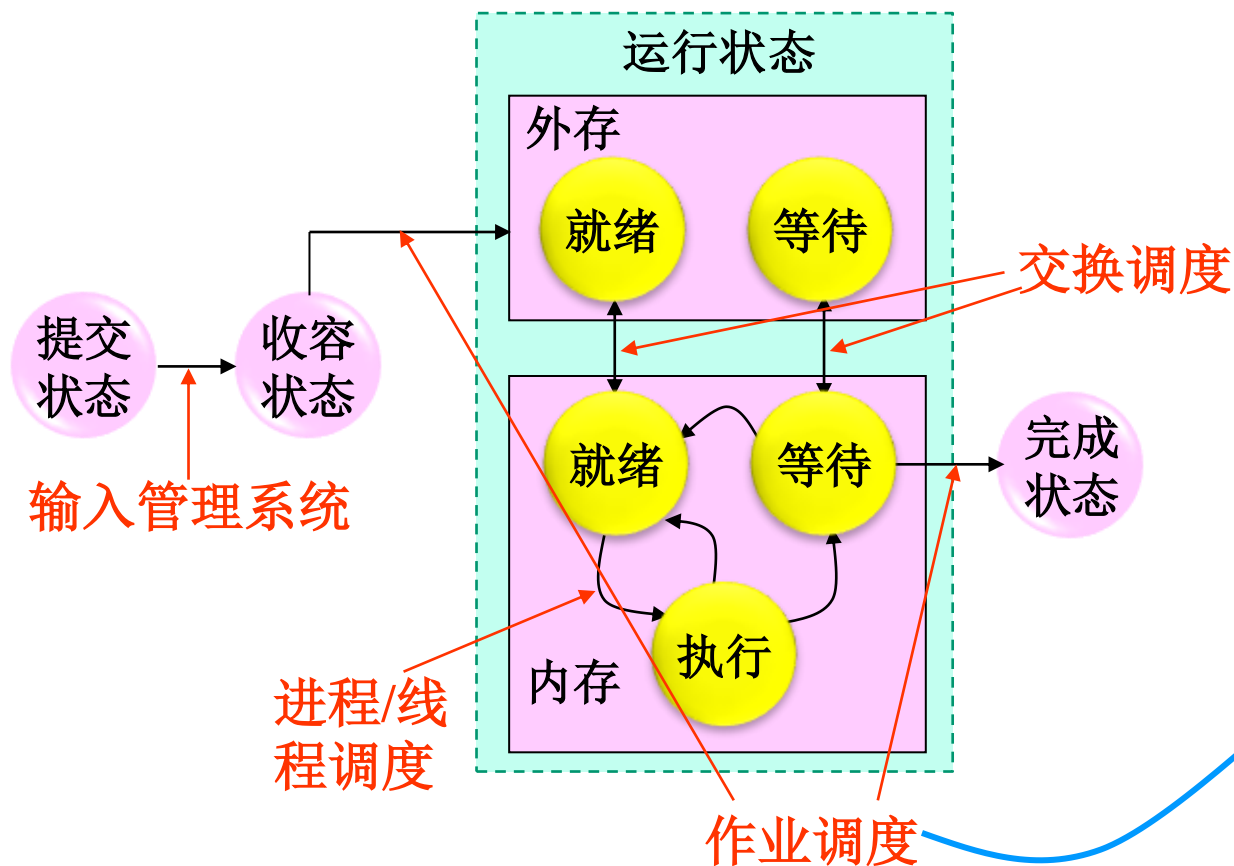
- **提交状态:** 作业处于从输入设备进入外存的过程;
- **收容(后备)状态:** 作业的全部信息被输入到输入井, 尚未被调度执行;
- **运行状态:** 作业被调度程序选中, 装入内存运行
- **完成状态:** 作业运行完毕, 所占资源尚未被收回。



7.1 分级调度

7.1.2 调度的层次

●四级：作业调度、交换调度、进程调度、线程调度



第1级：作业调度、宏观调度、高级调度

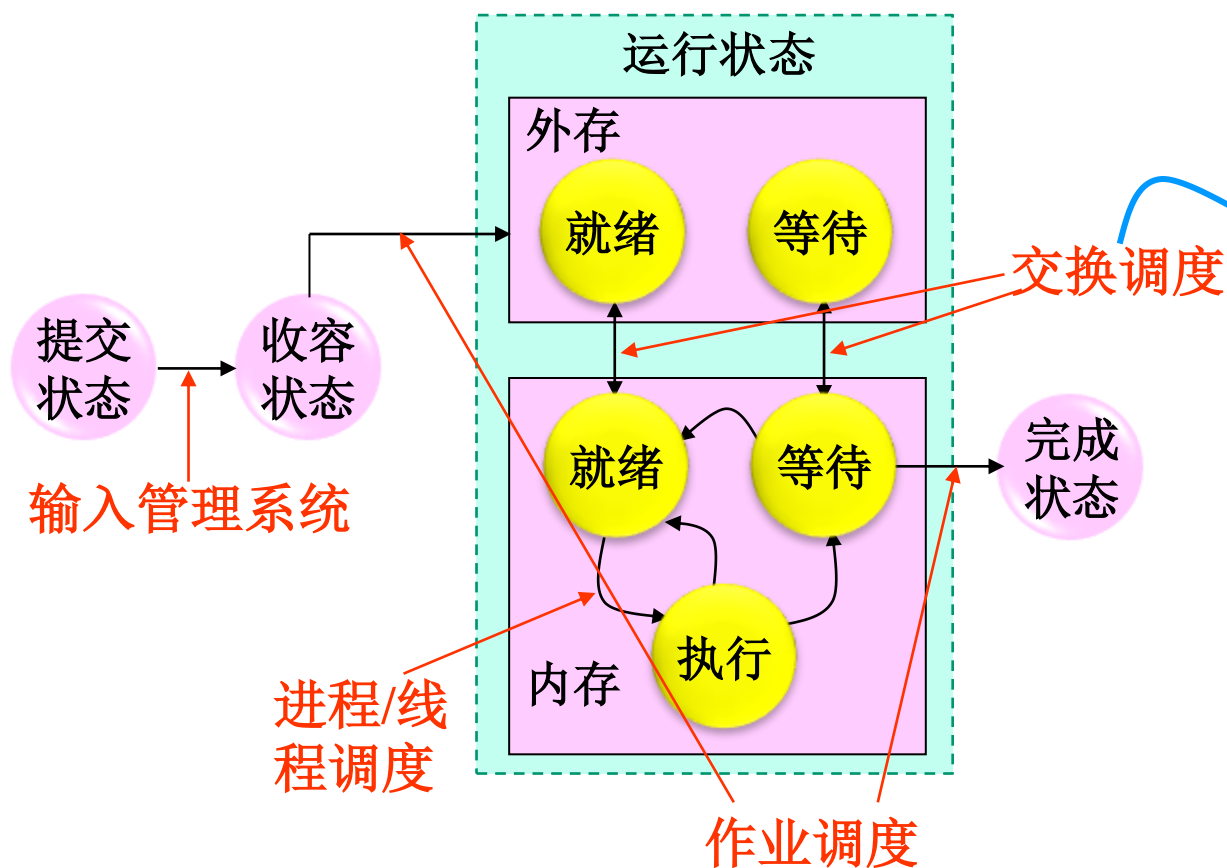
● 对外存输入井上的大量作业进行选择，对选择的作业分配资源，建立相应进程。作业执行完毕时，回收资源。



7.1 分级调度

7.1.2 调度的层次

- **四级：**作业调度、交换调度、进程调度、线程调度



第2级：交换调度、 中级调度

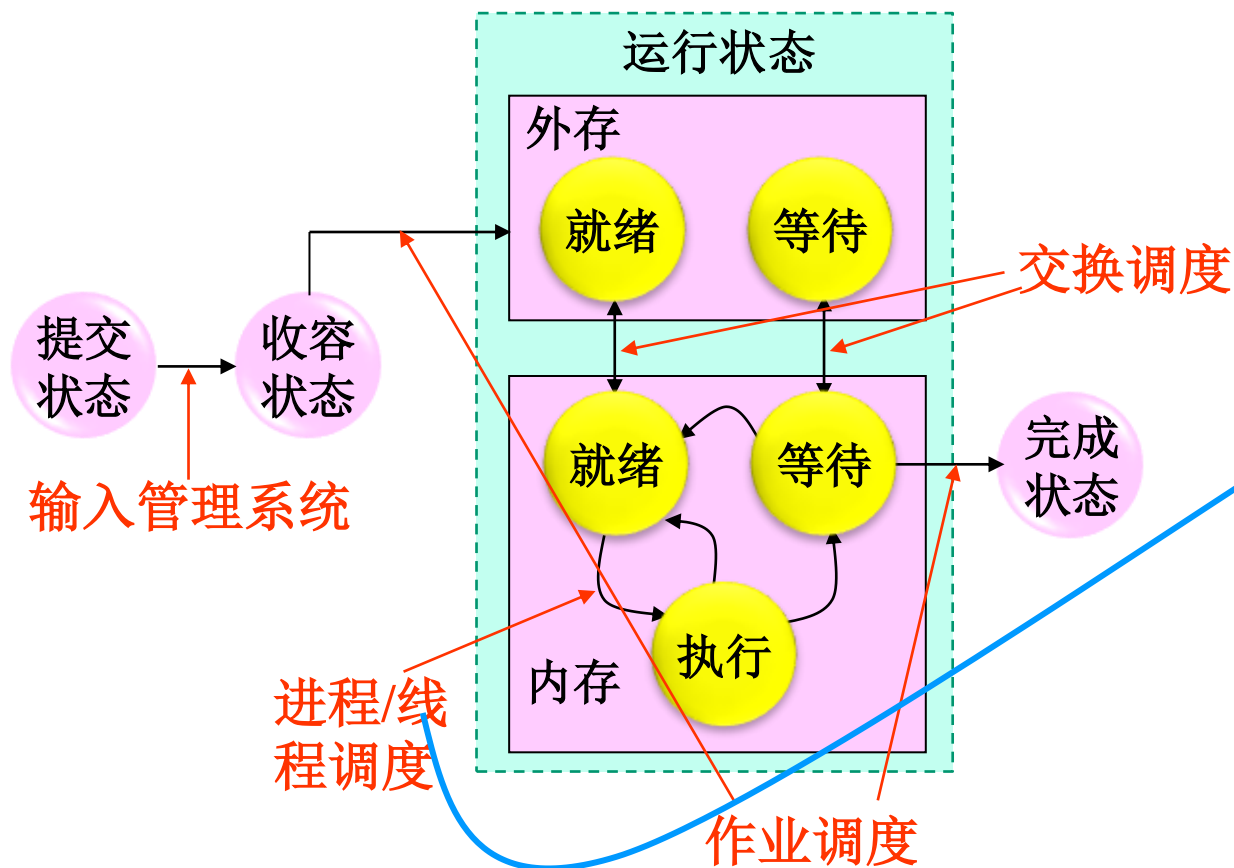
● 将处于外存交换区中的就绪状态或等待状态的进程调入内存，或把处于内存就绪状态或内存等待状态的进程交换到外存交换区。



7.1 分级调度

7.1.2 调度的层次

- 四级：作业调度、交换调度、进程调度、线程调度



第3级：进程调度、微观调度、低级调度

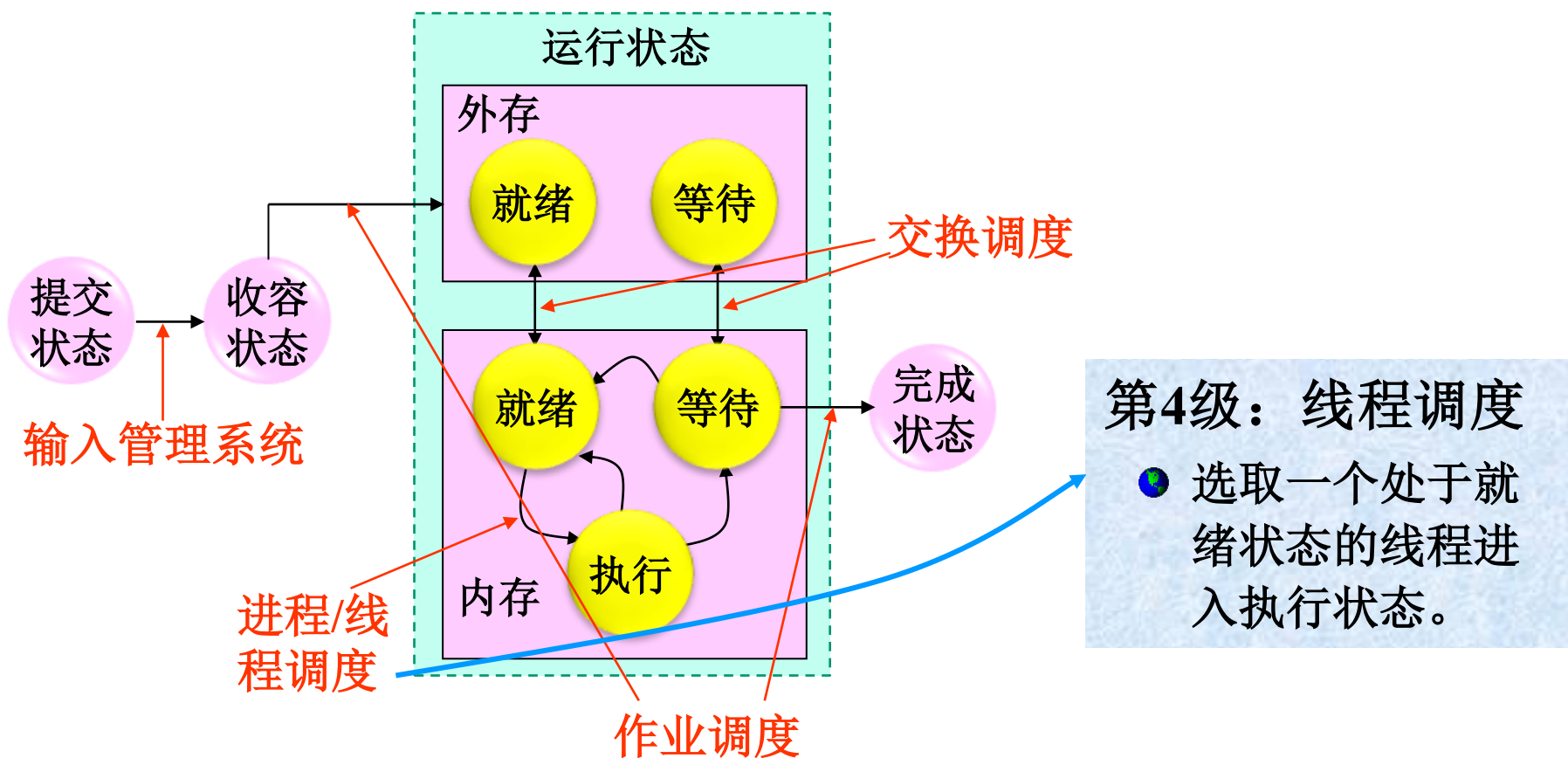
- 选取一个处于就绪状态的进程占用处理机，之后，进行上下文切换以便建立与占用处理机进程相适应的执行环境。



7.1 分级调度

7.1.2 调度的层次

- 四级：作业调度、交换调度、进程调度、线程调度

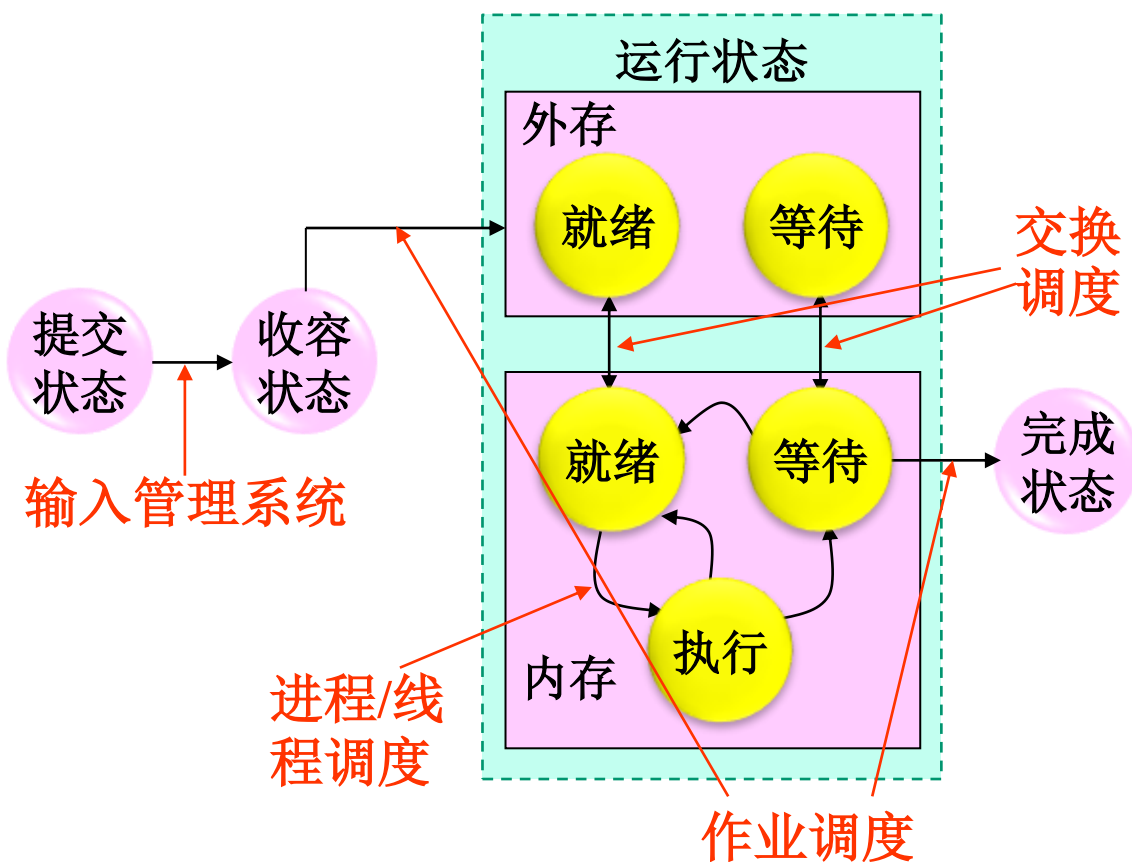




7.1 分级调度

7.1.2 调度的层次

问题：这四级调度，在一个操作系统中都必须存在吗？

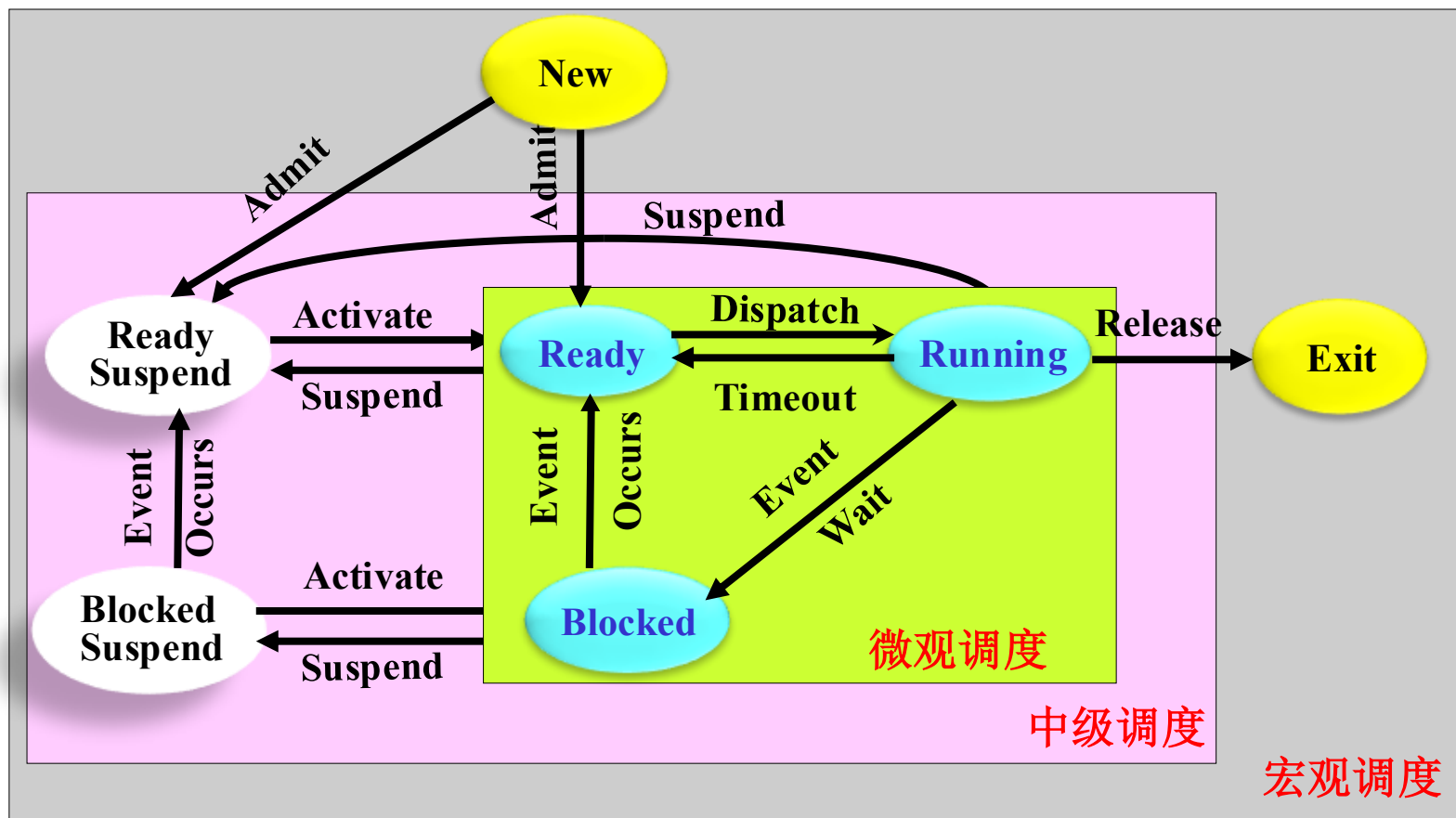


- 多道批处理系统存在作业调度和进程调度；
- 分时系统和实时系统一般只有进程调度、交换调度和线程调度。



7.1 分级调度

7.1.2 调度的层次





7.1 分级调度

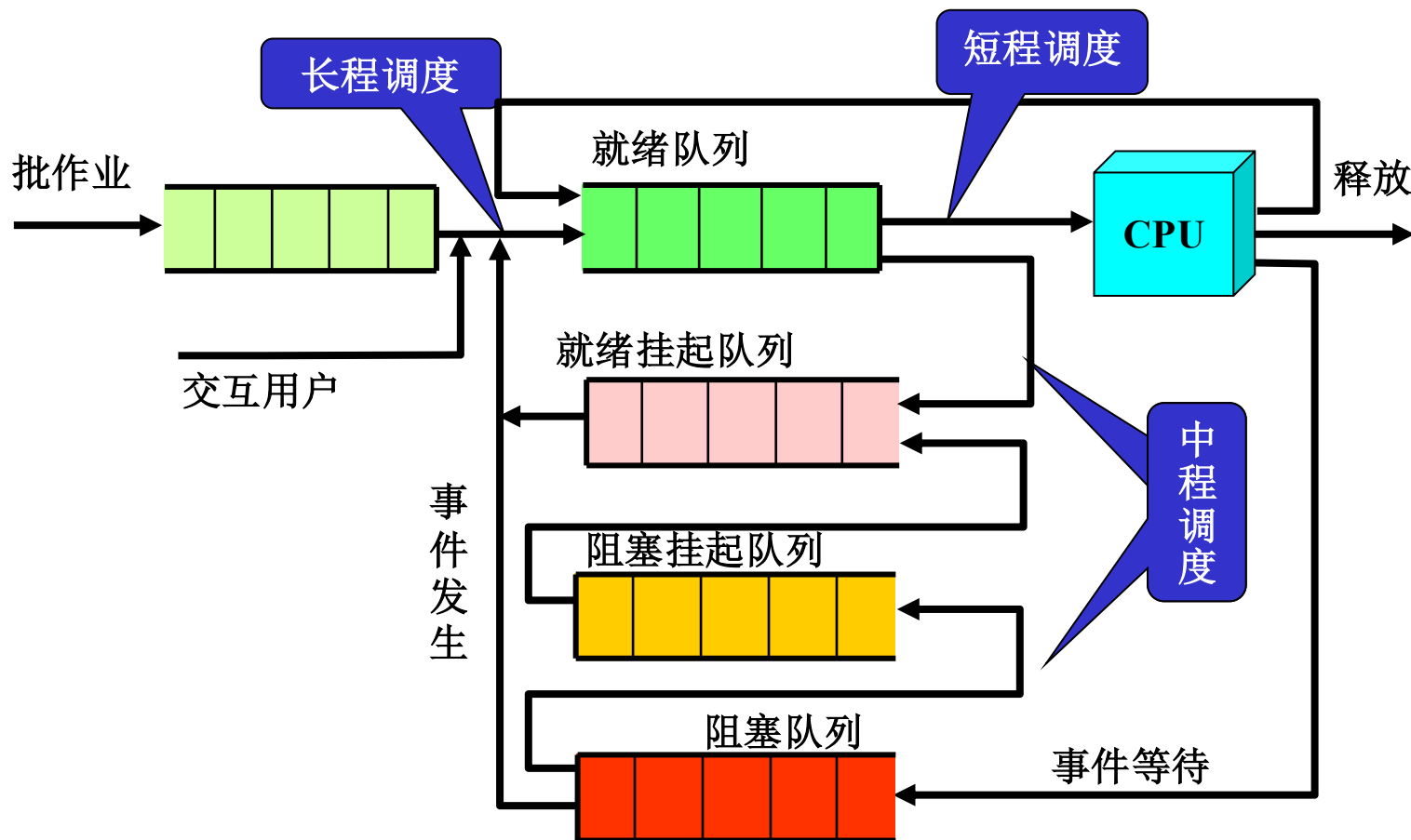
7.1.3 调度时间周期

- **长期(long-term)**: 将进程投入“允许执行”进程缓冲池中, 或送到“退出”进程缓冲池中。进程状态: **New** → **Ready** suspend (**Ready**), **Running** → **Exit**, 它将控制多道程序的程度, 执行频率相对较低;
- **中期(medium-term)**: 将进程的部分或全部加载到内存中。进程状态: **Ready** ↔ **Ready suspend**, **Blocked** ↔ **Blocked suspend** ;
- **短期(short-term)**: 也称分派程序 (**dispatcher**) 选择哪个进程在处理器上执行。进程状态: **Ready** ↔ **Running**;
- **I/O调度**: 选择哪个I/O等待进程, 使其请求可以被空闲的I/O设备进行处理。



7.1 分级调度

7.1.3 调度时间周期 用于调度的队列图





7.1 分级调度

7.1.4 OS类型

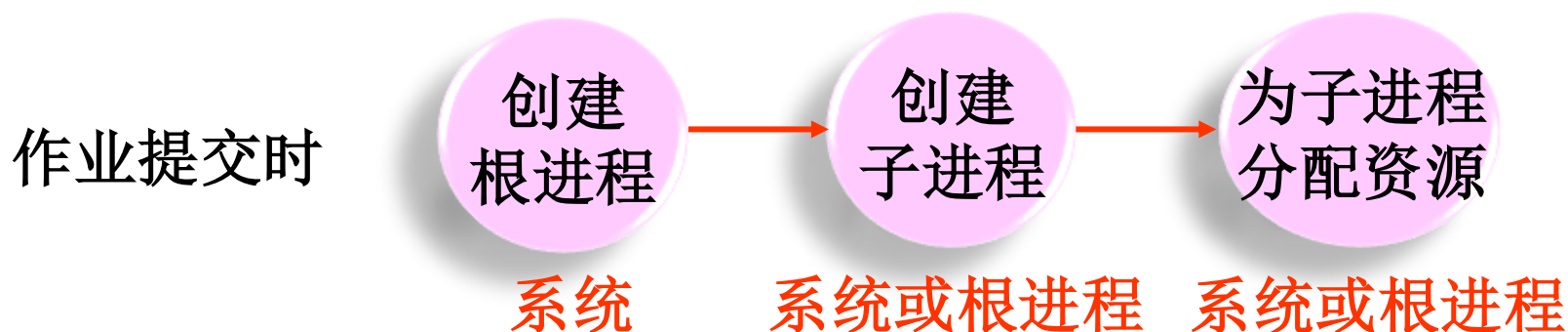
- **批处理调度**——应用场合：大中型主机集中计算，如工程计算、理论计算（流体力学）
- **分时调度、实时调度**：通常没有专门的作业调度
- **多处理机调度**



7.1 分级调度

7.1.5 作业、进程、线程的关系

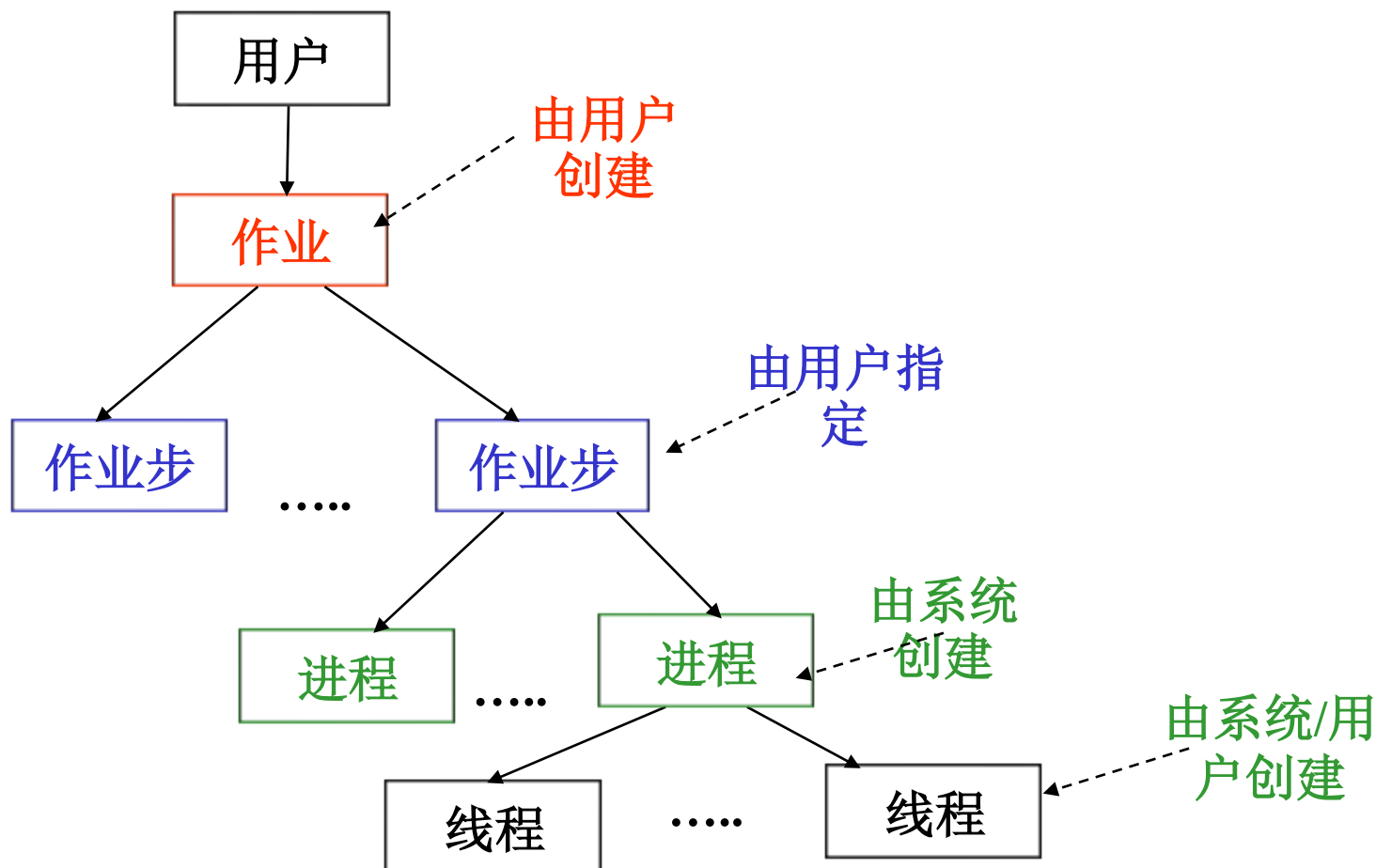
- 作业是用户向计算机提交任务的任务实体；
- 进程是计算机为完成用户任务而设置的执行实体，是系统分配资源的基本单位；
- 一个作业由多个进程组成。
- 线程占用CPU的基本单位





7.1 分级调度

7.1.5 作业、进程、线程的关系



1. 处理机调度可分为三级, 它们是高级调度, [填空1] 和低级调度; 在一般操作系统中, 必须具备的调度是 [填空2] .

作答

2、某计算机系统中 8 台打印机，有 K 个进程竞争使用，每个进程最多需要 3 台打印机。该系统可能会发生死锁的 K 的最小值是（ C ）

A 2

B 3

C 4

D 5

提交

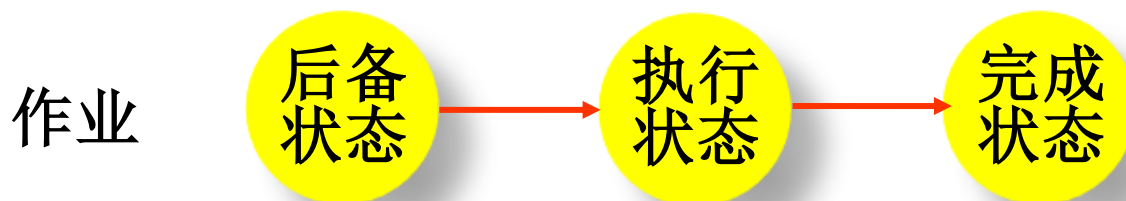
3. 通常不采用()方法来解除死锁.

- ☐ A 终止一个死锁进程
- ☐ B 终止所有死锁进程
- ☐ C 从死锁进程处抢夺资源
- ☒ D 从非死锁进程处抢夺资源

提交



7.2 作业调度概述



7.2.1 作业调度的功能

● 记录系统中各作业的状况

系统为每个作业建立一个**JCB**记录作业信息，系统通过**JCB**感知作业的存在。

作业进入后备状态时，系统为其建立**JCB**；作业进入完成状态后，系统撤销其**JCB**。

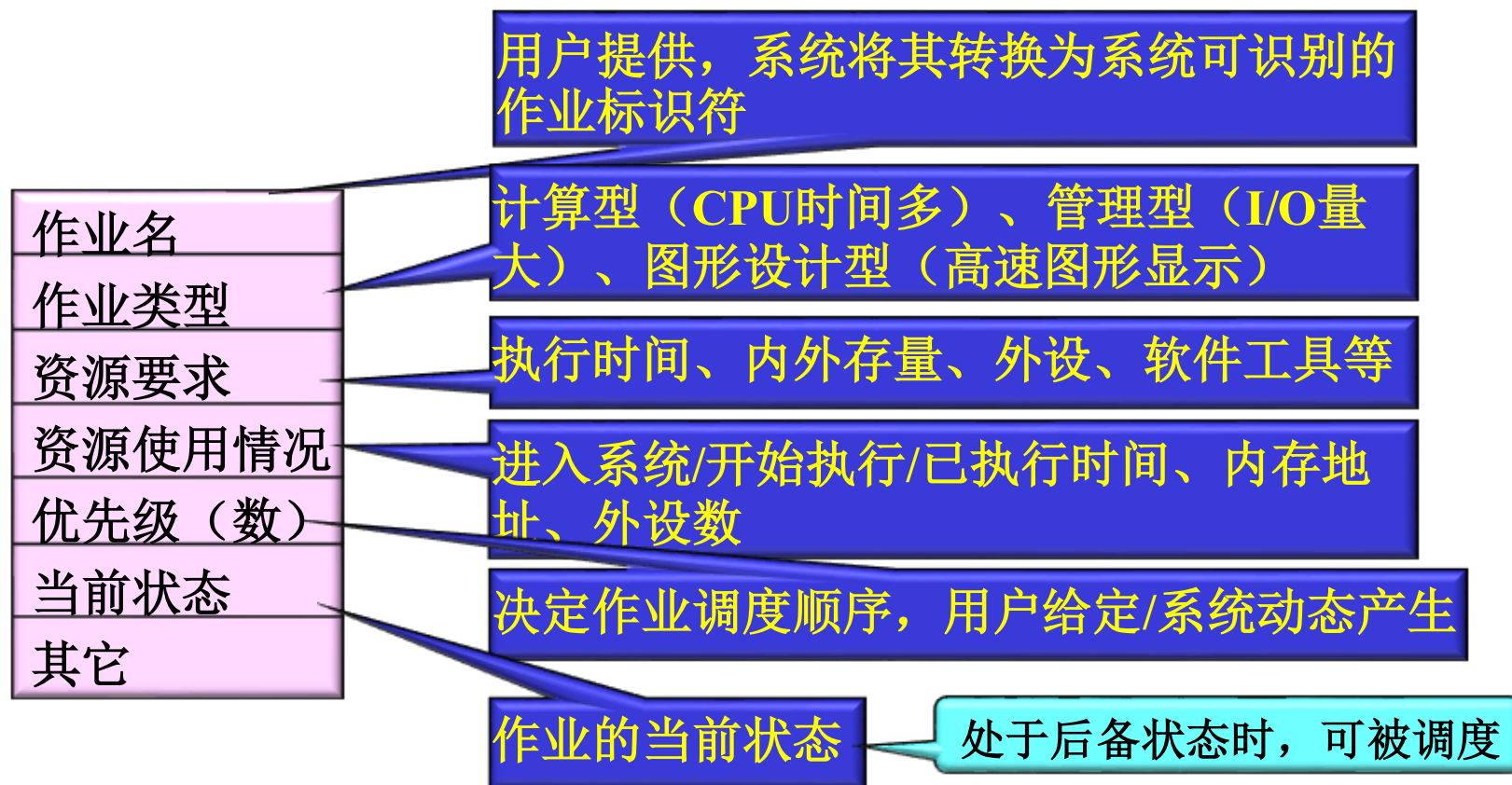
作业名
作业类型
资源要求
资源使用情况
优先级（数）
当前状态
其它



7.2 作业调度概述

7.2.1 作业调度的功能

● 记录系统中各作业的状况





7.2 作业调度概述

7.2.1 作业调度的功能

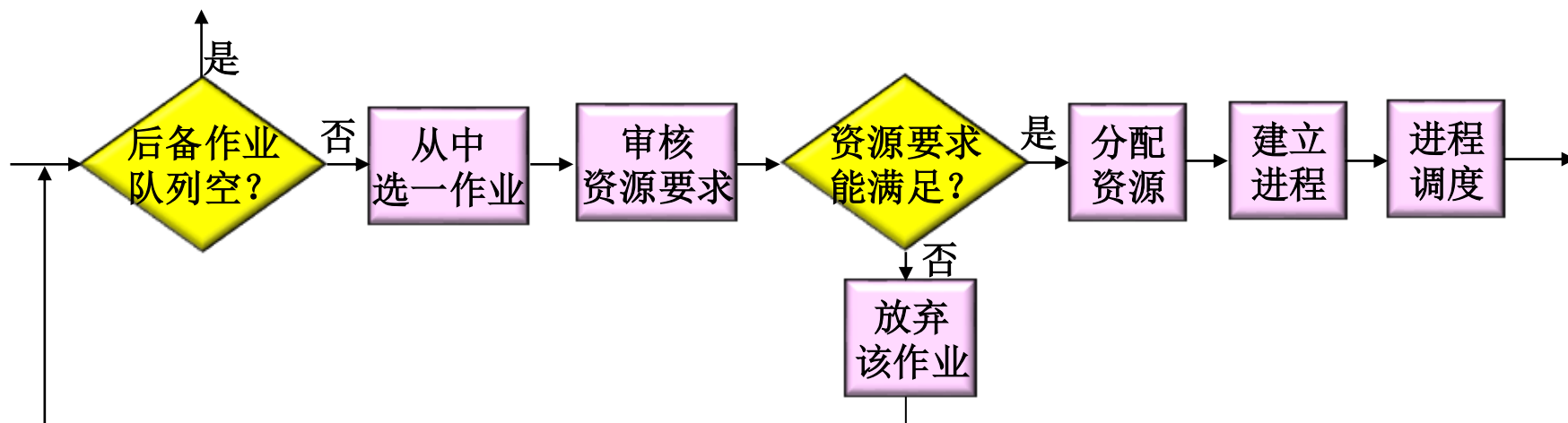
- ① 记录系统中各作业的状况
- ② 从后备队列中选择一部分作业投入运行
(涉及调度算法)
- ③ 为被选中的作业做好执行前的准备 (建立进程、为进程们分配系统资源)
- ④ 作业执行结束时的后处理



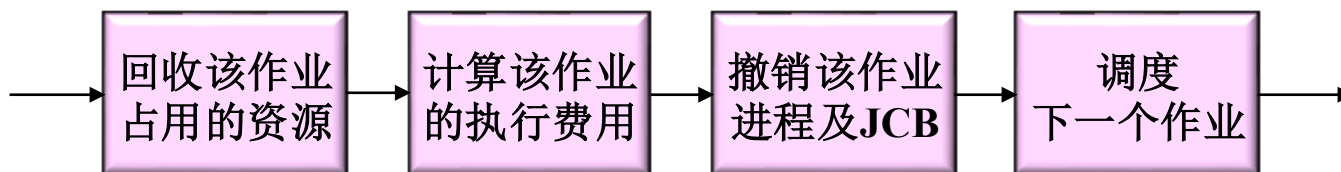
7.2 作业调度概述

7.2.2 作业调度中状态的转换

从后备状态到执行状态



从执行状态到完成状态





7.2 作业调度概述

7.2.3 作业调度目标与性能衡量

● 目标

- **公平性**: 对所有作业应该是公平的
- **利用率**: 应使设备有高的利用率
- **作业量**: 每天执行尽可能多的作业
- **响应时间**: 有快的响应时间



7.2 作业调度概述

7.2.3 作业调度目标与性能衡量

- **性能衡量：**可从不同的角度来判断处理机调度算法的性能，如：
 - 用户的角度
 - 处理机的角度
 - 算法实现的角度
- 实际的处理机调度算法选择是一个**综合的判断结果**



7.2 作业调度概述

7.2.3 作业调度目标与性能衡量

(1) 面向用户的调度性能准则

● **三个指标：** 周转时间、带权周转时间、响应时间

● **周转时间：** 作业从**提交到完成**（得到结果）所经历的时间。包括：在收容队列中等待，CPU上执行，就绪队列和阻塞队列中等待，结果输出等待——批处理系统

● **周转时间** T_i = 作业完成时间(T_{ei}) - 作业提交时间(T_{si})
= 作业等待时间(T_{wi}) + 作业执行时间(T_{ri})

● **平均周转时间**

$$T = \frac{1}{n} \sum_{i=1}^n T_i$$



7.2 作业调度概述

7.2.3 作业调度目标与性能衡量

(1) 面向用户的调度性能准则

● 带权周转时间

● 带权周转时间 $W_i = T_i / T_{ri}$

● 平均带权周转时间
$$W = \frac{1}{n} \sum_{i=1}^n W_i$$

● **响应时间**：用户输入一个请求（如击键）到系统给出首次响应（如屏幕显示）的时间——分时系统



7.2 作业调度概述

7.2.3 作业调度目标与性能衡量

(2) 面向系统的调度性能准则

- **吞吐量：**单位时间内所完成的作业数，与作业本身特性和调度算法都有关系——批处理系统
- **处理机利用率：**——大中型主机
- **各种设备的均衡利用：**如CPU繁忙的作业和I/O繁忙（指次数多，每次时间短）的作业搭配——大中型主机



7.2 作业调度概述

7.2.3 作业调度目标与性能衡量

(3) 算法本身的调度性能准则

- 易于执行
- 执行开销比



7.2 作业调度概述

7.2.4 小结

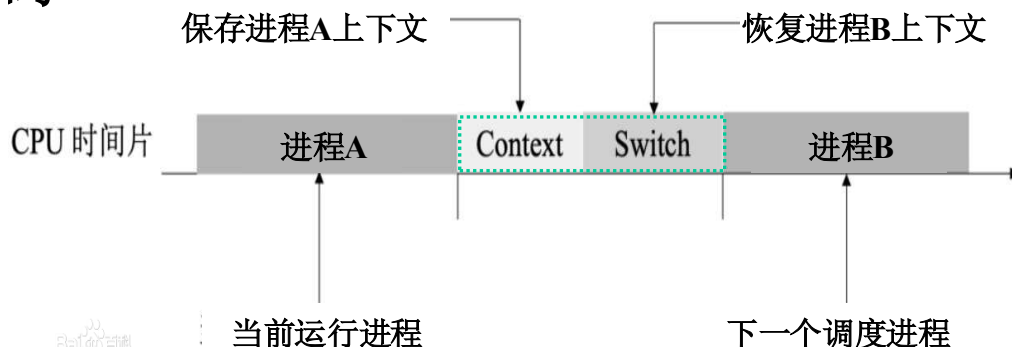
- 作业概念、组成、状态
- 作业建立：输入方式、JCB
- 作业调度
 - 功能
 - 目标
 - 作业调度的性能衡量：周转时间、平均周转时间
 - **重要概念：** 作业、JCB、（平均）周转时间



7.3 进程调度概述

7.3.1 进程调度的功能

- ① 记录所有进程的运行状况（静态和动态）
- ② 选择下一个要运行的合适进程：当进程出让CPU或调度程序剥夺执行状态进程占用的CPU时，选择适当的进程分派CPU；
- ③ 完成上下文切换：用户态执行进程A通过时钟中断或系统调用进入OS核心的进程调度器，完成：
 - 保存进程A的上下文，恢复进程B的上下文（CPU寄存器和一些表格的当前指针
 - 用户态执行进程B代码
 - 上下文切换之后，指令和数据快速缓存cache通常需要更新，执行速度降低。





7.3 进程调度概述

7.3.2 进程调度的时机

OS在以下几种原因之一发生时进行进程调度

- ① 正在执行的进程**执行完毕**
- ② 分时系统中**时间片用完**
- ③ 执行进程**自己调用阻塞原语**使自己变为等待状态
- ④ 执行进程**调用P操作**，因资源不足被阻塞
- ⑤ 执行进程因**I/O被阻塞**
- ⑥ 将睡眠的**进程唤醒**，将其加入就绪队列后，执行进程调度程序
- ⑦ 可剥夺方式下，就绪队列中某进程优先级**高于**执行进程
- ⑧ 系统调用执行完毕，**从系统程序返回到用户程序**时，进行进程调度

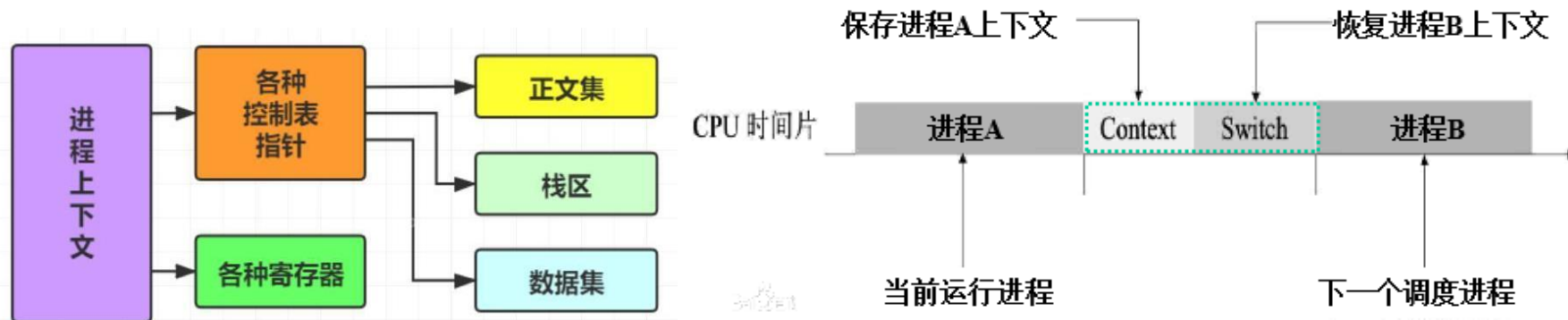
C
P
U
空
闲



7.3 进程调度概述

7.3.3 进程上下文切换

- 进程上下文由正文段、数据段、硬件寄存器的内容以及有关数据结构组成
- 进程上下文切换步骤
 - ① 决定是否切换以及是否允许切换
 - ② 保存当前执行进程的上下文
 - ③ 恢复或装配所选进程的上下文，将CPU控制权交给所选中进程





7.3 进程调度概述

7.3.4 进程调度性能评价

● 定性衡量

- ✓ 调度的可靠性
- ✓ 调度的简洁性

● 定量衡量

- ✓ CPU利用率
- ✓ 进程在队列中的等待时间与执行时间之比



7.4 调度算法

有多个**作业位于后备（或收容）队列**，有多个**进程处于就绪状态**，如何选择下一个要运行的作业或进程？

普通的作业/进程/线程调度算法：

1. 先来先服务（**First Come First Service**）
2. 短作业优先（**Shortest Job First**）
3. 最高响应比优先（**Highest Response Ratio Next**）
7. 时间片轮转法（**Round Robin**）
5. 多级队列算法（**Mutiple Level Queue**）
6. 优先级算法（**Priority**）
7. 多级反馈队列算法（**Round Robin with Multiple Feedback**）



7.4 调度算法

7.4.1 先来先服务（FCFS, First Come First Service）

● 原理：

- 按照作业提交或进程变为就绪状态的先后次序，分派CPU；
- 当前作业或进程占用CPU，直到执行完或阻塞，才出让CPU（非抢占方式）；
- 在作业或进程唤醒后（如I/O完成），并不立即恢复执行，通常等到当前作业或进程出让CPU

● 特点（评价一下）：

- 简单。最简单的算法
- 有利于？不利于？

● 适用于：作业、进程、线程的调度

A.长作业, B.短作业, C.CPU繁忙的作业, D.I/O繁忙的作业



7.4 调度算法

7.4.1 先来先服务：按就绪的先后顺序进行调度

- 例：假设在**单道批处理环境下**有四个作业，已知它们进入系统的时间、估计运行时间：

作业	进入时间	估计运行时间（分钟）	开始运行时间	结束运行时间	周转时间
JOB1	8:00	120			
JOB2	8:50	50			
JOB3	9:00	10			
JOB4	9:50	20			
平均周转时间					

- 问题：FCFS调度执行序列是什么？
✓ 执行序列：J1 --> J2 --> J3 --> J4
- 思考：与开始调度的时间是什么关系？周转时间多少？
- ◆ 问题：如何改进FCFS算法？



7.4 调度算法

7.4.2 短作业优先 (SJF, Shortest Job First)

又称为“短进程优先” SPN(Shortest Process Next);
是对FCFS算法的改进，其目标是减少平均周转时间

● SJF算法原理：

- 选择**预计执行时间**短的作业（进程）优先分派处理机。
- 通常后来的短作业**不抢先**正在执行的作业



7.4 调度算法

7.4.2 短作业优先 (SJF, Shortest Job First)

● SJF优点

- 比FCFS改善平均周转时间和平均带权周转时间，缩短作业的等待时间
- 提高系统的吞吐量

● SJF缺点？

- 对长作业非常不利，可能长时间得不到执行
- 未能依据作业的紧迫程度来划分执行的优先级
- 难以准确估计作业（进程）的执行时间，从而影响调度性能



7.4 调度算法

7.4.2 短作业优先 (SJF, Shortest Job First)

- 例：假设在单道批处理环境下有四个作业，已知它们进入系统的时间、估计运行时间：

作业	进入时间	估计运行时间（分钟）	开始运行时间	结束运行时间	周转时间
JOB1	8:00	120			
JOB2	8:50	50			
JOB3	9:00	10			
JOB4	9:50	20			
平均周转时间					

- 问题：SJF调度执行序列是什么？
 - ✓ 执行序列：J3还是J1？ -->J? -->J? -->J?
 - ? 思考：与开始调度的时间有关系吗？



7.4 调度算法

7.4.2 短作业优先 (SJF, Shortest Job First)

🌐 SJF变形

- ① 最短剩余时间优先SRT(Shortest Remaining Time):
允许比当前进程剩余时间更短的进程来抢占
- ② 最高响应比优先HRRN(Highest Response Ratio Next):
是FCFS和SJF的折衷。



7.4 调度算法

7.4.3 最高响应比优先 (HRRN)

- **原理**：选择响应比最高的作业优先启动。
- 响应比 $R = \text{作业周转时间 } T_i / \text{作业处理时间 } T_{ri}$
$$= (\text{作业处理时间 } T_{ri} + \text{作业等待时间 } T_{wi}) / \text{作业处理时间 } T_{ri}$$
$$= 1 + (\text{作业等待时间 } T_{wi} / \text{作业处理时间 } T_{ri})$$
- 该算法是FCFS和SJF的结合，克服了两种算法的缺点
 - **优点**：公平，吞吐率大
 - **缺点**：增加了计算，增加了开销



7.4 调度算法

7.4.3 最高响应比优先 (HRRN)

- 例：假设在单道批处理环境下有四个作业，已知它们进入系统的时间、估计运行时间：

作业	进入时间	估计运行时间（分钟）
JOB1	8:00	120
JOB2	8:50	50
JOB3	9:00	10
JOB4	9:50	20

- 问题：HRRN调度执行序列是什么？
 - ✓ 设8:00开始调度（与开始调度时间有关）
 - ✓ J1（只有它到达） $\rightarrow (, 1+70/50, 1+60/10, 1+10/20)$
 - ✓ J1、J3, $\rightarrow (, 1+80/50, , 1+20/20)$, 选J2
 - ✓ J1,J3,J2 $\rightarrow (, , , 1+70/20)$, 选J4



Review



1. 分级调度

- 作业调度（宏观调度、长程）、交换调度（中级调度、中程）、进程/线程调度（微观调度、短程）

2. 作业调度概述

- 功能、目标、衡量指标（周转时间、平均周转时间、带权周转时间、平均带权周转时间、响应时间）

3. 进程调度概述

- 功能、目标、时机、进程上下文切换

4. 调度算法

- FCFS、SJF、HRRN



7.4 调度算法

7.4.4 时间片轮转算法 (Round Robin)

● 说明

- 前三种算法主要用于**宏观调度**，说明怎样选择一个进程或作业开始运行，开始运行后的作法都相同，即运行到结束或阻塞，阻塞结束时等待当前进程放弃CPU；
- 本算法主要用于**微观调度**，说明怎样并发运行，即**切换的方式**；设计目标是提高**资源利用率**；
- 其基本思路是通过**时间片轮转**，提高进程**并发性**和**响应时间**特性，从而提高**资源利用率**。

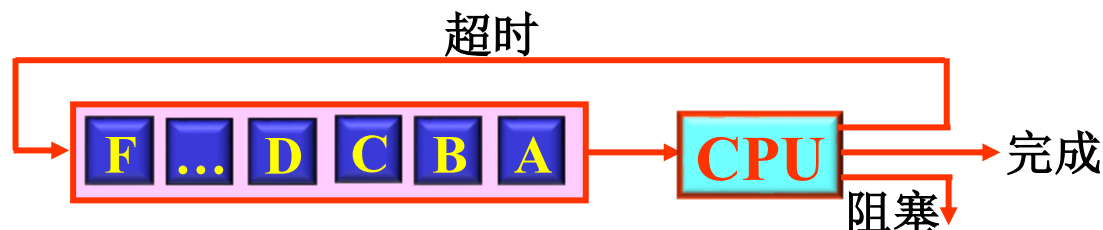


7.4 调度算法

7.4.4 时间片轮转算法 (Round Robin)

● Round Robin算法

- 将系统中所有的就绪进程按照FCFS原则，排成一个**队列**
- 每次调度时将CPU分派给**队首进程**，让其**执行一个时间片**。时间片的**长度**从几个ms到几百ms
- 在一个时间片结束时，**时钟中断检测到时间片用完**；
- 据此调用调度程序**暂停当前进程的执行**，将其送到**就绪队列末尾**，并通过上下文切换**执行当前的队首进程**
- 进程可以**未使用完一个时间片**，就**出让CPU**（如阻塞）





7.4 调度算法

7.4.4 时间片轮转算法 (Round Robin)

● 时间片长度的确定

➤ 时间片长度变化的影响

● **过长**⇒退化为FCFS算法，进程在一个时间片内都执行完，响应时间长；

● **过短**⇒用户的一次请求需要多个时间片才能处理完，上下文切换次数增加，响应时间长。

➤ 对响应时间的要求

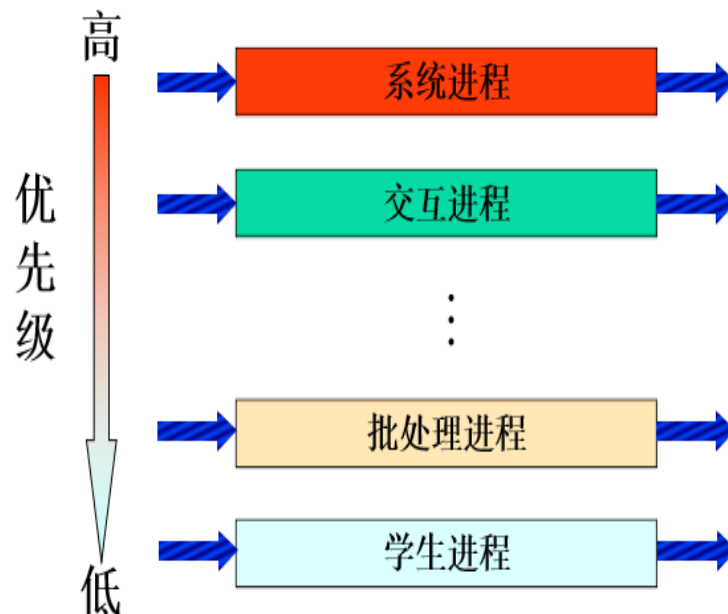
● $T(\text{响应时间}) = N(\text{进程数目}) * q(\text{时间片})$



7.4 调度算法

7.4.5 多级队列算法 (Multiple-level Queue)

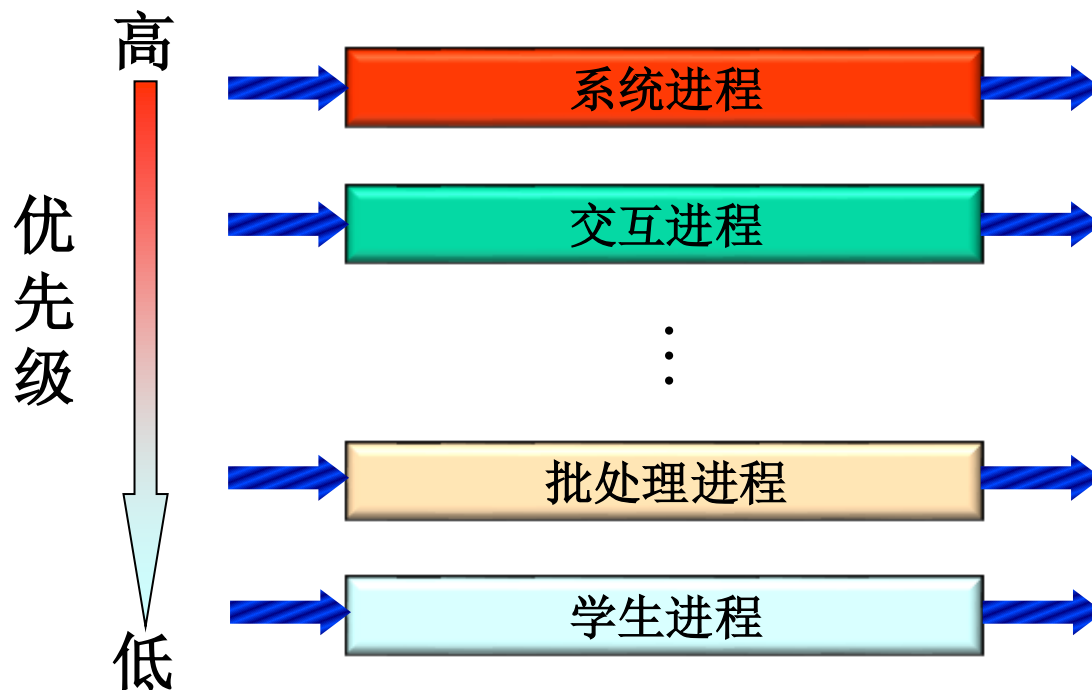
- 本算法引入多个就绪队列，通过各队列的区别对待，达到一个综合的调度目标
- 回想：FCFS、RR，只设一个就绪队列
- 多级队列算法
 - 根据作业或进程性质或类型的不同，将就绪队列再分为若干个**子队列**；
 - 每个作业或进程固定归入一个队列
 - 各队列的**不同处理**：不同队列可有**不同的优先级、时间片长度、调度策略**等。如：系统进程、用户交互进程、批处理进程等。





7.4 调度算法

7.4.5 多级队列算法 (Multiple-level Queue)



调度策略：按优先级从高到低，执行完高优先级队列中的进程，再执行低一级队列中的进程，直到最低优先级队列；同一队列中的进程按照FCFS算法调度。



7.4 调度算法

7.4.6 优先级算法

是多级队列算法的改进，平衡各进程对响应时间的要求。
适用于作业调度和进程调度，可分成**抢先式**和**非抢先式**

● 静态优先级

- **定义**：创建进程时就确定，直到进程终止前都不改变。
通常是一个整数。
- **依据**
 - **进程类型**（系统进程优先级较高）；
 - **对资源的需求**（对CPU和内存需求较少的进程，
优先级较高）；
 - **用户要求**（紧迫程度和付费多少）。



7.4 调度算法

7.4.6 优先级算法

● 动态优先级

- 在创建进程时赋予的优先级，在进程运行过程中可以自动改变，以便获得更好的调度性能。如：
 - 在就绪队列中，**等待时间**延长则**提高**优先级，从而使优先级较低的进程在等待足够的时间后，其优先级提高到可被调度执行；
 - 进程每**执行一个时间片**，就**降低**其优先级，从而一个进程持续执行时，其优先级降低到出让CPU



7.4 调度算法

7.4.6 优先级算法

● 线性优先级调度 (SRR, Selfish Round Robin)

- 是优先级算法的一个实例，它通过**进程优先级的递增**来改进长执行时间进程的周转时间特征

● SRR算法

➤ 就绪进程队列分成两个：**新创建进程队列**和**享受服务队列**

- **新创建进程队列**：按FCFS方式排成；进程优先级按速率**a**增加；
- **享受服务队列**：已得到过时间片服务的进程按FCFS方式排成；进程优先级按速率**b**增加。
- **新创建进程等待时间的确定**：当新创建进程优先级与享受服务队列中最后一个进程优先级相同时，转入享受服务队列，**前提是什么**？





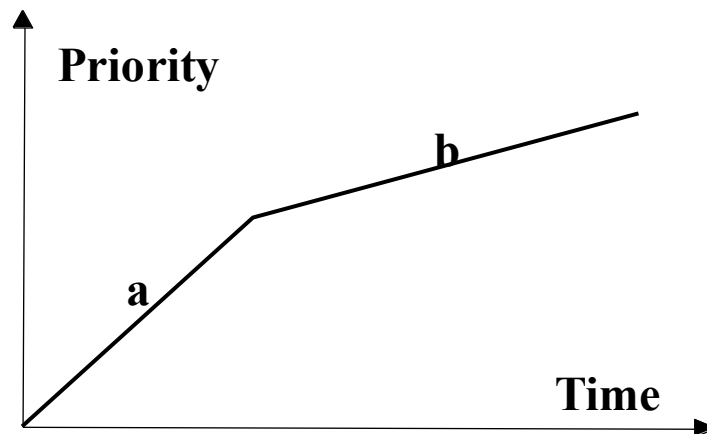
7.4 调度算法

7.4.6 优先级算法

● 线性优先级调度 (**SRR, Selfish Round Robin**)

● **思考:** a 与 b 的大小关系, 如何影响SRR算法?

- 当 $b > a > 0$ 时, 享受服务队列中永远只有一个进程; SRR算法退化成FCFS算法;
- 当 $a > b = 0$ 时, SRR算法就是时间片轮转算法。



SRR算法的优先级变化

1. 作业调度是从输入井中处于()状态的作业中选取作业调入主存运行.

- ☐ A 运行
- ☒ B 收容
- ☐ C 输入
- ☐ D 就绪

提交

2.一种既有利于短小作业又兼顾到长作业的作业调度算法是(C)

- ☐ A 先来先服务
- ☐ B 短作业优先
- ☒ C 最高响应比优先
- ☐ D 均衡调度

提交

3. 时间片轮转法进行进程调度是为了()。

- ☒ A 多个终端都能得到系统的及时响应
- ☐ B 先来先服务
- ☐ C 优先级较高的进程得到及时响应
- ☐ D 需要cpu最短的进程先做

提交

此题未设置答案，请点击右侧设置按钮

4. 我们如果为每一个作业只建立一个进程，则为了照顾短作业用户，应采用()；为照顾紧急作业用户，应采用()，为能实现人机交互作用应采用()，而能使短作业，长作业及交互作业用户都比较满意时，应采用()。

A

FCFS调度算法

B

短作业优先调度算法

C

时间片轮转法

D

多级反馈队列调度算法

E

基于优先权的剥夺调度算法

F

响应比优先算法

提交

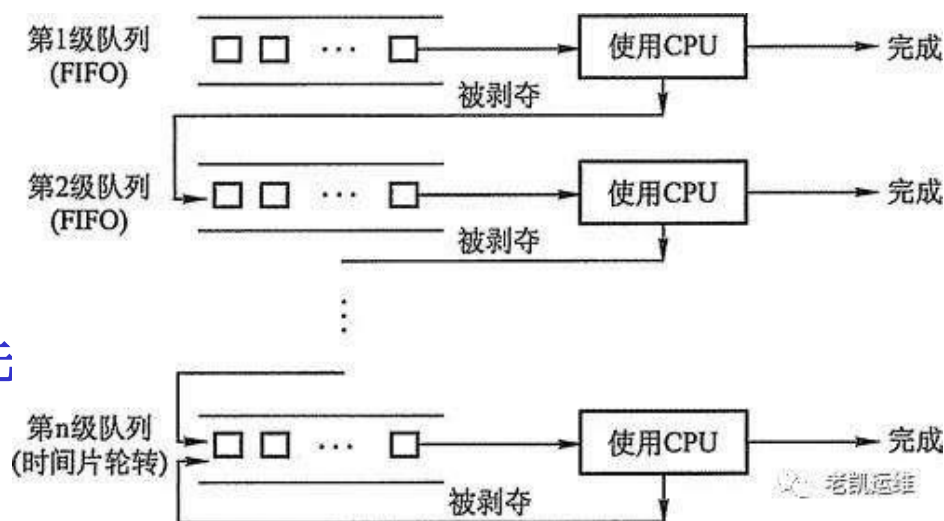


7.4 调度算法

7.4.7 多级反馈队列算法 (Round Robin with Multiple Feedback)

● 多级反馈队列算法

- 设置多个就绪队列，分别赋予不同的优先级，如逐级降低，队列1的优先级最高。每个队列执行时间片的长度也不同，规定优先级越低则时间片越长，如逐级加倍；
- 新进程进入内存后，先投入队列1的末尾，按FCFS算法调度；若按队列1一个时间片未能执行完，则降低投入到队列2的末尾，同样按FCFS算法调度；如此下去，降低到最后的队列，则按RR算法调度直到完成；
- 仅当较高优先级的队列为空，才调度较低优先级的队列中的进程执行。
- 如果进程执行时有新进程进入较高优先级的队列，则抢先执行新进程，并把被抢先的进程投入原队列的末尾。

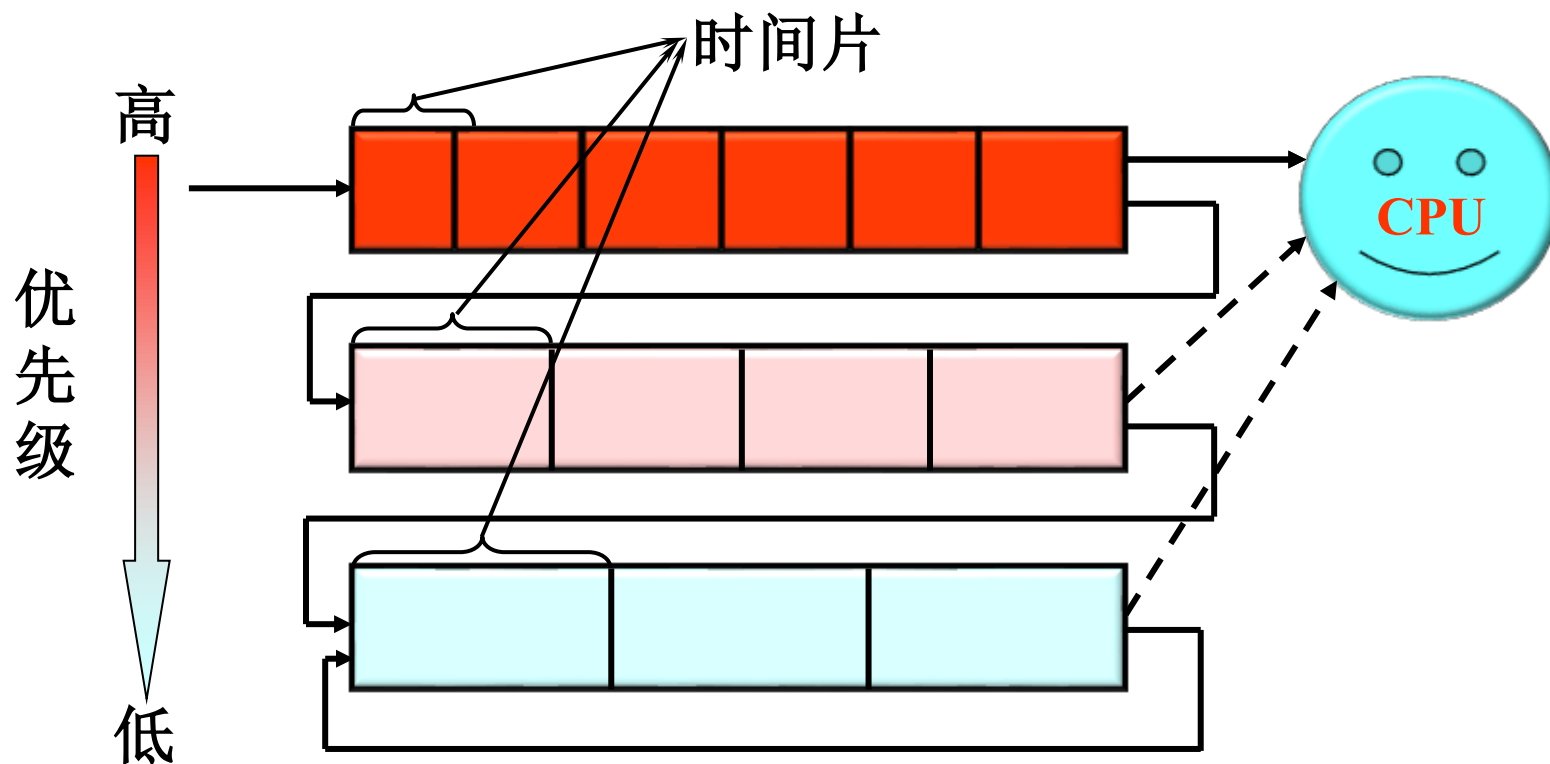




7.4 调度算法

7.4.7 多级反馈队列算法 (Round Robin with Multiple Feedback)

● 多级反馈队列

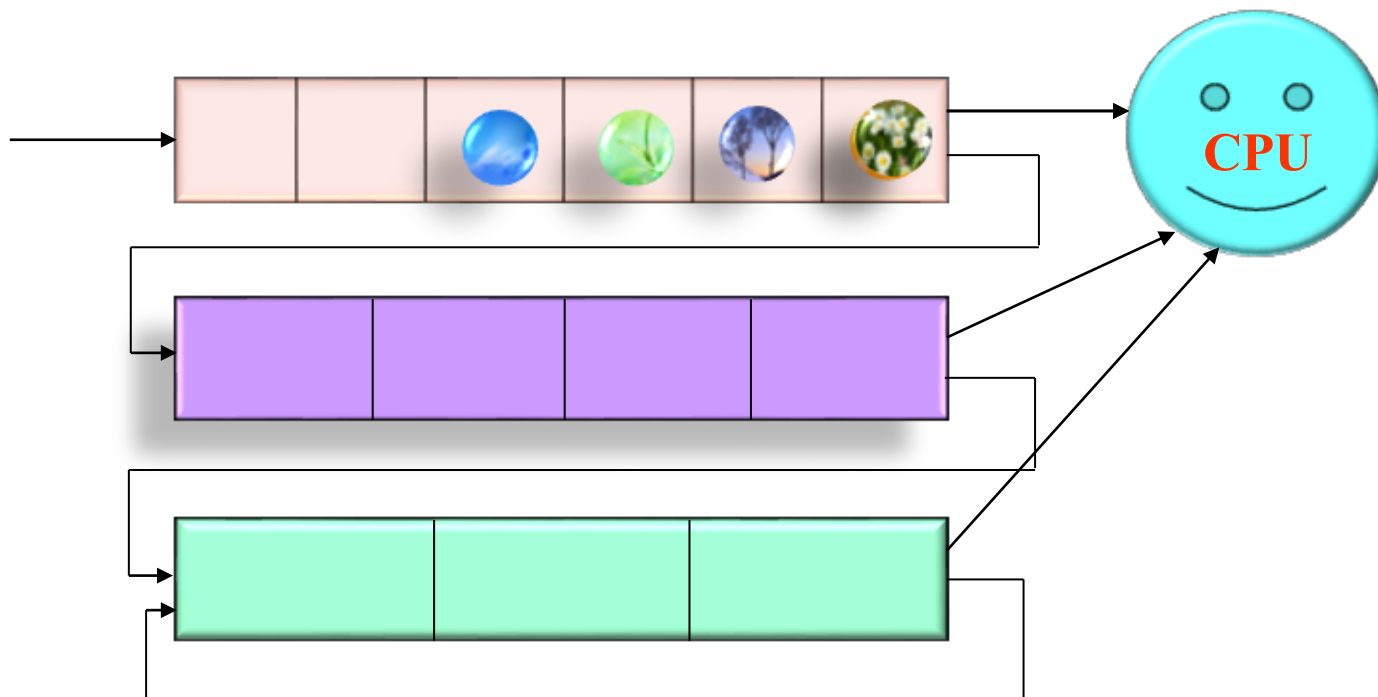




7.4 调度算法

7.4.7 多级反馈队列算法 (Round Robin with Multiple Feedback)

● 多级反馈队列





7.4 调度算法

7.4.7 多级反馈队列算法 (Round Robin with Multiple Feedback)

●是时间片轮转算法和优先级算法的综合和发展。

优点:

- 为提高系统吞吐量和缩短平均周转时间而照顾短进程
- 为获得较好的I/O设备利用率和缩短响应时间而照顾I/O型进程
- 不必估计进程的执行时间，动态调节



7.4 调度算法

7.4.7 多级反馈队列算法 (Round Robin with Multiple Feedback)

● 几点说明

- **I/O型进程**：让其进入最高优先级队列，以**及时响应**I/O交互。通常执行一个时间片，要求可**处理完一次I/O请求**的数据，然后转入到阻塞队列；
- **计算型进程**：每次都执行完时间片，进入更低级队列。最终采用最大时间片来执行，**减少调度**；
- **I/O次数不多，而主要是CPU处理的进程**：在I/O完成后，放回原先I/O请求时离开的队列，以免每次都回到最高优先级队列后再逐次下降；
- 为适应一个进程**在不同时间段的运行特点**，I/O完成时（I/O次数多的情形），**提高**优先级；时间片用完时，**降低**优先级。



7.4 调度算法

7.4.8 调度算法性能指标

● FCFS, Round Robin, SRR周转时间比较

- 长作业时： $T(\text{FCFS}) < T(\text{SRR}) < T(\text{RR})$ （运行时间是主要因素）；
- 短作业时： $T(\text{RR}) < T(\text{SRR}) < T(\text{FCFS})$ （等待时间是主要因素）。



7.5 调度算法应用举例

- **例1:** 假设在**单道批处理**环境下有4个作业，已知它们进入系统的时间、估计运行时间：

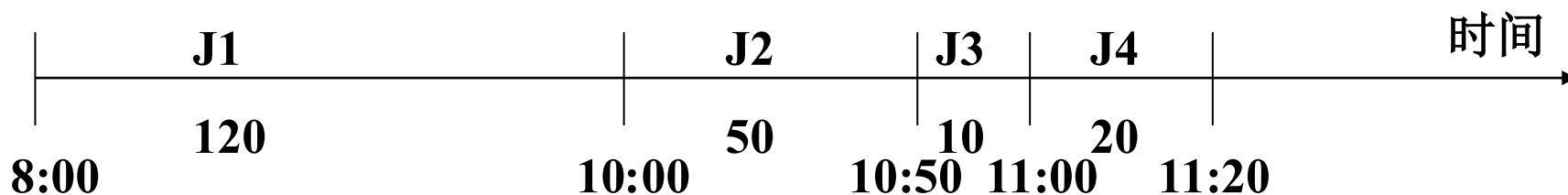
作业	进入时间	估计运行时间（分钟）
JOB1	8:00	120
JOB2	8:50	50
JOB3	9:00	10
JOB4	9:50	20

- 应用先来先服务(FCFS)、最短作业优先(SJF)和最高响应比优先(HRRN)作业调度算法，分别计算出作业的平均周转时间和带权平均周转时间，假设从第一个作业到达时开始调度。



7.5 调度算法应用举例

● 先来先服务算法 (FCFS) 计算结果

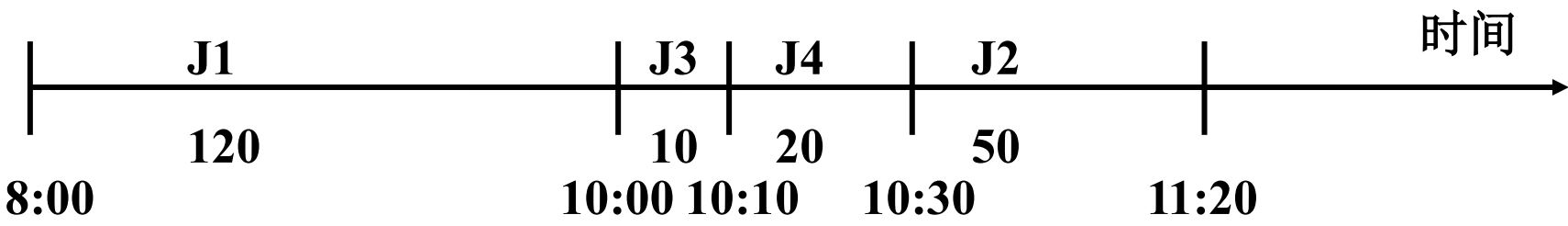


作业	进入时间	估计运行时间 (分钟)	开始时间	结束时间	周转时间 (分钟)	带权周转时间
JOB1	8: 00	120	8: 00	10: 00	120	1
JOB2	8: 50	50	10: 00	10: 50	120	2.4
JOB3	9: 00	10	10: 50	11: 00	120	12
JOB4	9: 50	20	11: 00	11: 20	90	4.5
作业平均周转时间 $T = 112.5$ 分钟 作业平均带权周转时间 $W = 4.975$					450	19.9



7.5 调度算法应用举例

最短作业优先算法计算结果

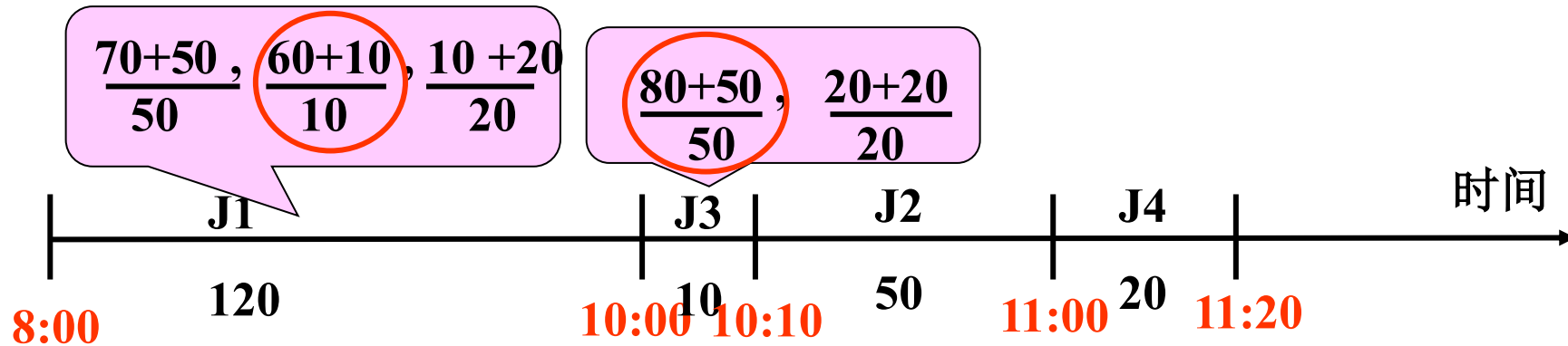


作业	进入时间	估计运行 时间 (分钟)	开始时间	结束时间	周转时间 (分钟)	带权周转 时间
JOB1	8: 00	120	8: 00	10: 00	120	1
JOB2	8: 50	50	10: 30	11: 20	150	3
JOB3	9: 00	10	10: 00	10: 10	70	7
JOB4	9: 50	20	10: 10	10: 30	40	2
作业平均周转时间 $T = 95$ 作业带权平均周转时间 $W = 3.25$					380	13



7.5 调度算法应用举例

最高响应比算法计算结果



作业	进入时间	估计运行 时间 (分钟)	开始时间	结束时间	周转时间 (分钟)	带权周转 时间
JOB1	8: 00	120	8: 00	10: 00	120	1
JOB2	8: 50	50	10: 10	11: 00	130	2.6
JOB3	9: 00	10	10: 00	10: 10	70	7
JOB4	9: 50	20	11: 00	11: 20	90	4.5
作业平均周转时间 T = 102.5 作业带权平均周转时间 W = 3.775					410	15.1



7.5 调度算法应用举例

- **例2：**在**两道**环境下有**四个作业**，已知它们进入系统的时间、估计运行时间，作业调度采用**短作业优先**算法，作业被调度运行后不再退出，进程调度采用**剩余时间最短优先**算法（假设一个作业创建一个进程）；
- 请给出这四个作业的执行时间序列，并计算出平均周转时间及带权平均周转时间。



7.5 调度算法应用举例

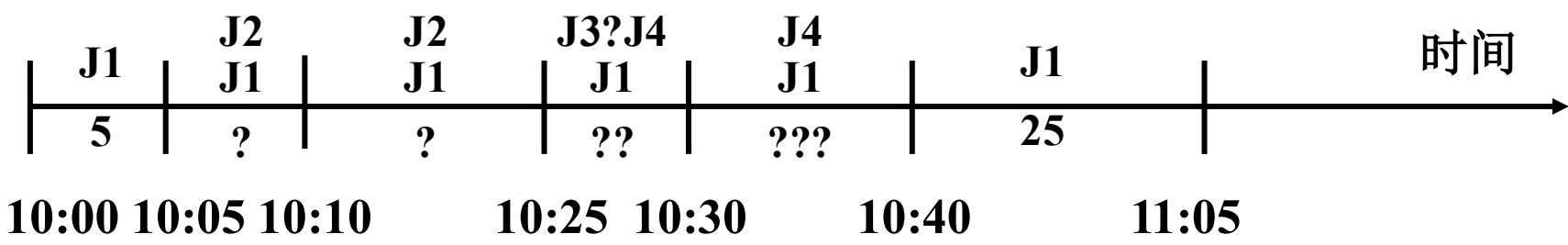
●最短作业优先算法执行结果（两道环境）

作业	进入时间	估计运行时间 (分钟)
JOB1	10: 00	30
JOB2	10: 05	20
JOB3	10: 10	5
JOB4	10: 20	10



7.5 调度算法应用举例

最短作业优先算法执行结果（两道环境）



作业	进入时间	估计运行时间 (分钟)	开始时间	结束时间	周转时间 (分钟)	带权周转时间
JOB1	10: 00	30	10: 00	11: 05	65	2.167
JOB2	10: 05	20	10: 05	10: 25	20	1
JOB3	10: 10	5	10: 25	10: 30	20	4
JOB4	10: 20	10	10: 30	10: 40	20	2
作业平均周转时间 T = 31.25 作业平均带权周转时间 W = 2.29					125	9.167



7.5 调度算法应用举例

● 最短作业优先算法执行分析过程

- 10: 00, JOB1进入, 只有一作业, JOB1被调入执行,
10: 05, JOB2到达, 最多允许两作业同时进入, 所以
JOB2也被调入;
- 内存中有两作业, 哪一个执行? 题目规定当一新作业运行后, 可按作业运行时间长短调整执行次序;
- 即基于优先数可抢占式调度策略, 优先数是根据作业估计运行时间大小来决定的, 由于JOB2运行时间(20分)比JOB1少(到10: 05, JOB1还需25分钟), 所以JOB2运行, 而JOB1等待。



7.5 调度算法应用举例

7.5 调度算法应用举例

●最短作业优先算法执行分析过程

- 10: 10, JOB3到达输入井, 内存已有两作业, JOB3不能马上进入内存; 10: 20, JOB4也不能进入内存, 10: 25, JOB2运行结束, 退出, 内存中剩下JOB1, 输入井中有两作业JOB3和JOB4, 如何调度?
- 作业调度算法: 最短作业优先, 因此JOB3进入内存, 比较JOB1和JOB3运行时间, JOB3运行时间短, 故JOB3运行, 同样, JOB3退出后, 下一个是JOB4, JOB4结束后, JOB1才能继续运行。



7.5 调度算法应用举例

7.5 调度算法应用举例

●最短作业优先算法执行分析过程

●四个作业的执行时间序列为：

JOB1: 10: 00—10: 05, 10: 40—11: 05

JOB2: 10: 05—10: 25

JOB3: 10: 25—10: 30

JOB4: 10: 30—10: 40





7.5 调度算法小结

7.5 调度算法应用举例

●最短作业优先算法执行分析过程

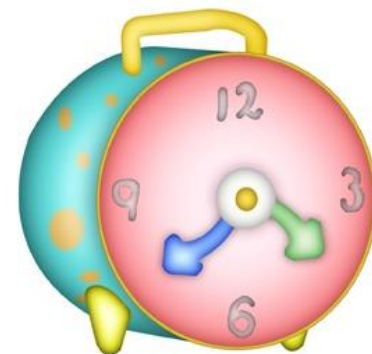
●四个作业的执行时间序列为：

JOB1: 10: 00—10: 05, 10: 40—11: 05

JOB2: 10: 05—10: 25

JOB3: 10: 25—10: 30

JOB4: 10: 30—10: 40





7.6 实时调度

7.6.1 实时调度概述

- **实时计算**越来越重要，实时调度是实时系统中最重要的组件；
- 实时系统，如实时控制、过程控制、机器人、远程通信等；
- **实时计算**系统的正确性不仅依赖于**计算的逻辑结果**，而且依赖于**产生结果的时间**；
- 通常给一个特定的任务制定一个**最后期限**，最后期限指定**开始时间或结束时间**。



7.6 实时调度

7.6.2 实时调度分类

● 按是否必须满足截止期，分为：

- 硬实时任务：必须满足最后期限的限制，否则产生致命的错误；
- 软实时任务：希望满足最后期限，但并不是强制的，超过期限，调度完成任务仍有意义；

● 按是否周期执行，分为：

- 一个非周期任务（aperiodic [₁ˌeɪpɪrɪˈɒdɪk] task）有一个必须结束或开始的最后期限；
- 周期任务，期限描述为“每隔周期T一次”。



7.6 实时调度

7.6.3 实时调度的依据

●提供必要的调度信息

- **就绪时间**：该任务成为就绪状态的起始时间；
- **开始或完成截止时间**：对于典型的实时应用，只需知道其一；
- **处理时间**：任务从开始执行直至完成所需时间；
- **资源要求**：任务执行时所需的一组资源；
- **绝对或相对优先级**（硬实时任务或软实时任务）。



7.6 实时调度

7.6.4 实时调度的特点--强

● 系统处理能力应足够强

- 设系统中有 m 个周期性的硬实时任务，它们的处理时间为 C_i ，周期时间为 P_i ，

- 单处理机时的限制条件

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1$$

- 多（ N ）处理机时的限制条件

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq N$$

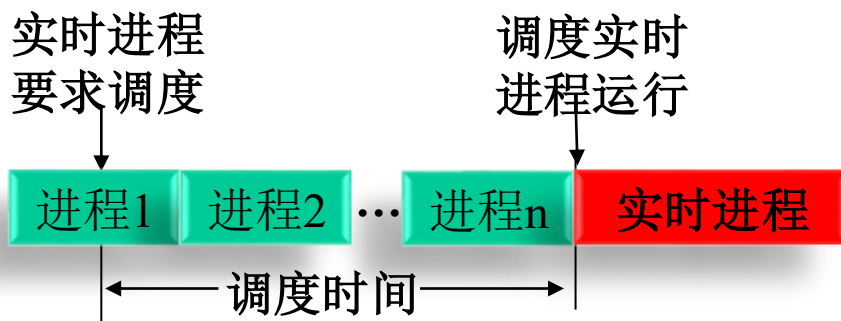


7.6 实时调度

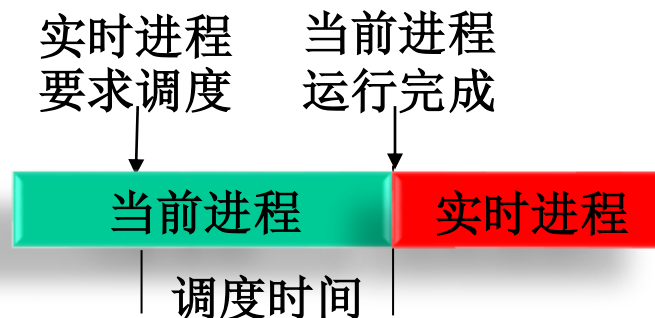
7.6.4 实时调度的特点-抢占

●采用抢先式调度

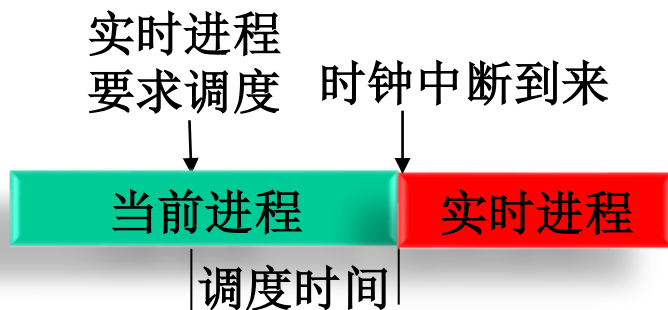
(1) 非抢占轮转调度



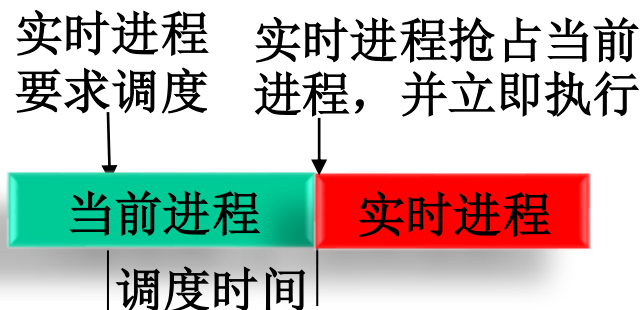
(2) 非抢占优先权调度



(3) 基于时钟中断抢占的优先权抢占调度



(4) 立即抢占的优先权调度





7.6 实时调度

7.6.4 实时调度的特点-抢占

1. 非抢占式调度算法

- (1) **非抢占式轮转调度算法**：由一台计算机控制着若干个相同的（或类似的）对象，为每一个被控对象建立一个实时任务，并将它们排成一个轮转队列。调度程序每次选择队列中的第一个任务投入运行。当该任务完成后，便把它挂在轮转队列的末尾等待，调度程序再选择下一个队首任务运行。这种调度算法可获得数秒至数十秒的响应时间，可用于要求不太严格的实时控制系统中。
- (2) **非抢占式优先调度算法**：如果在系统中还含有少数具有一定要求的实时任务，则可采用非抢占式优先调度算法，系统为这些任务赋予了较高的优先级。当这些实时任务到达时，把它们安排在就绪队列的队首，等待当前任务自我终止或运行完成后，便可去调度执行队首的高优先进程。这种调度算法在做了精心的处理后有可能使其响应时间减少到数秒至数百毫秒，因而可用于有一定要求的实时控制系统中。

2. 抢占式调度算法

- (3) **基于时钟中断的抢占式优先级调度算法**：在某实时任务到达后，如果它的优先级高于当前任务的优先级，这时并不立即抢占当前任务的处理机，而是等到时钟中断发生时，调度程序才剥夺当前任务的执行，将处理机分配给新到的高优先级任务。该算法能获得较好的响应效果，其调度延迟可降为几十至几毫秒，可用于大多数的实时系统中。
- (4) **立即抢占（Immediate Preemption）的优先级调度算法**：在这种调度策略中，要求操作系统具有快速响应外部事件中断的能力。一旦出现外部中断，只要当前任务未处于临界区，便能立即剥夺当前任务的执行，把处理机分配给请求中断的紧迫任务。这种算法能获得非常快的响应，可把调度延迟降低到几毫秒至100微秒，甚至更低。

https://blog.csdn.net/weixin_45798993/article/details/122576815



7.6 实时调度

7.6.4 实时调度的特点--快

- 快速中断响应，在中断处理时（硬件）关中断的时间尽量短；
- 快速上下文切换：相应地采用较小的调度单位（如线程）。



7.6 实时调度

7.6.5 实时调度算法

- ① 静态表驱动调度（Static table-driven scheduling）；
- ② 静态优先级驱动的抢占调度（Static priority-driven preemptive [pri'emptiv] scheduling）；
- ③ 动态分析调度算法（Dynamic planning-based scheduling）；
- ④ 无保障动态调度算法（动态最大努力调度Dynamic best effort scheduling）。



7.6 实时调度

7.6.6 静态表驱动调度算法 (Static table-driven scheduling)

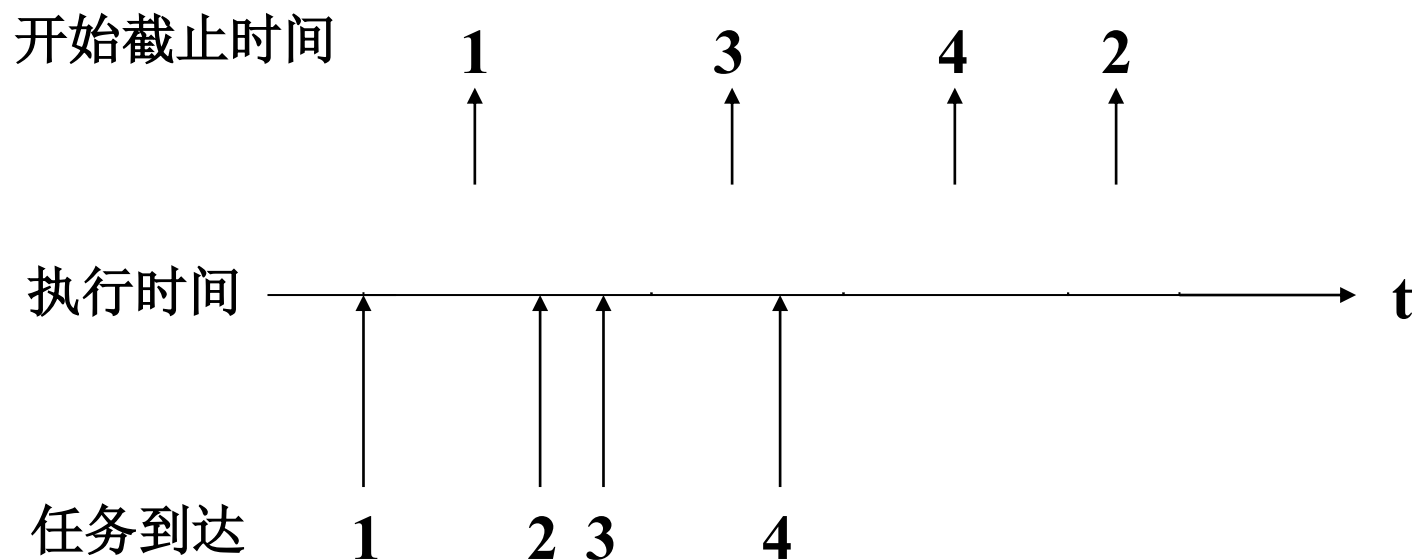
- **原理：**通过对所有周期性任务的输入信息分析预测，**事先**确定一个**固定**的调度方案，使得能够满足所有周期性任务的要求；
- **算法输入为：**到达时间、运行时间、最后结束时间、任务间的优先关系；
- **特点：**有效，但不灵活。因为任何任务要求的任何变化都需要重做调度；
- 适用于**周期性的实时应用**；
- **变种：**最早最后期限优先。



7.6 实时调度

7.6.6 静态表驱动调度算法 (Static table-driven scheduling)

● 最早最后 (截止) 期限优先 (EDF, Earliest Deadline First)





7.6 实时调度

7.6.7 静态优先级驱动的抢占调度 (Static priority-driven preemptive scheduling)

- 把通用的优先级调度算法用于实时系统;
- 优先级的确定是通过静态分析 (运行时间、到达频率) 完成的。即任务的优先级与每个任务的时间约束相关;
- 代表算法: 速率单调算法 (RMS-rate monotonic Scheduling) :



7.6 实时调度

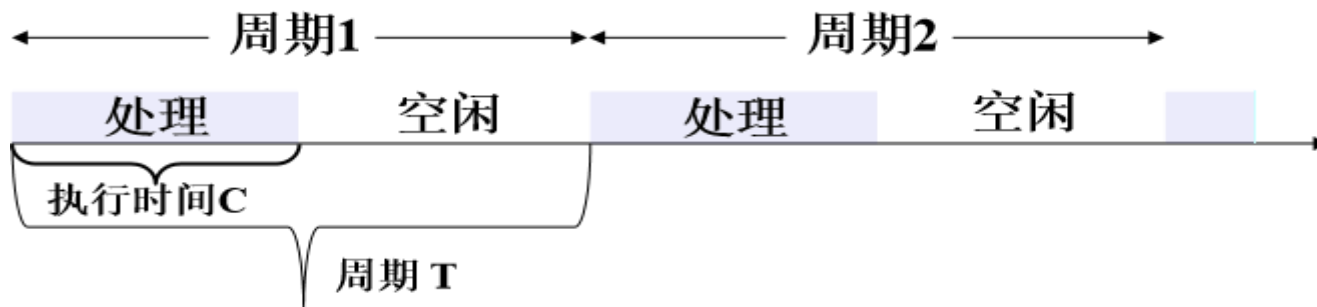
7.6.7 静态优先级驱动的抢占调度 (Static priority-driven preemptive scheduling)

● 速率单调算法 (RMS-rate monotonic Scheduling) :

- 任务的优先级 = 任务周期的倒数，即任务速率
- 优先级最高的任务是周期最短的任务

● 所有任务都满足最后期限则必须满足下式的限制

$$C_1/T_1 + C_2/T_2 + \dots + C_n/T_n \leq 1$$



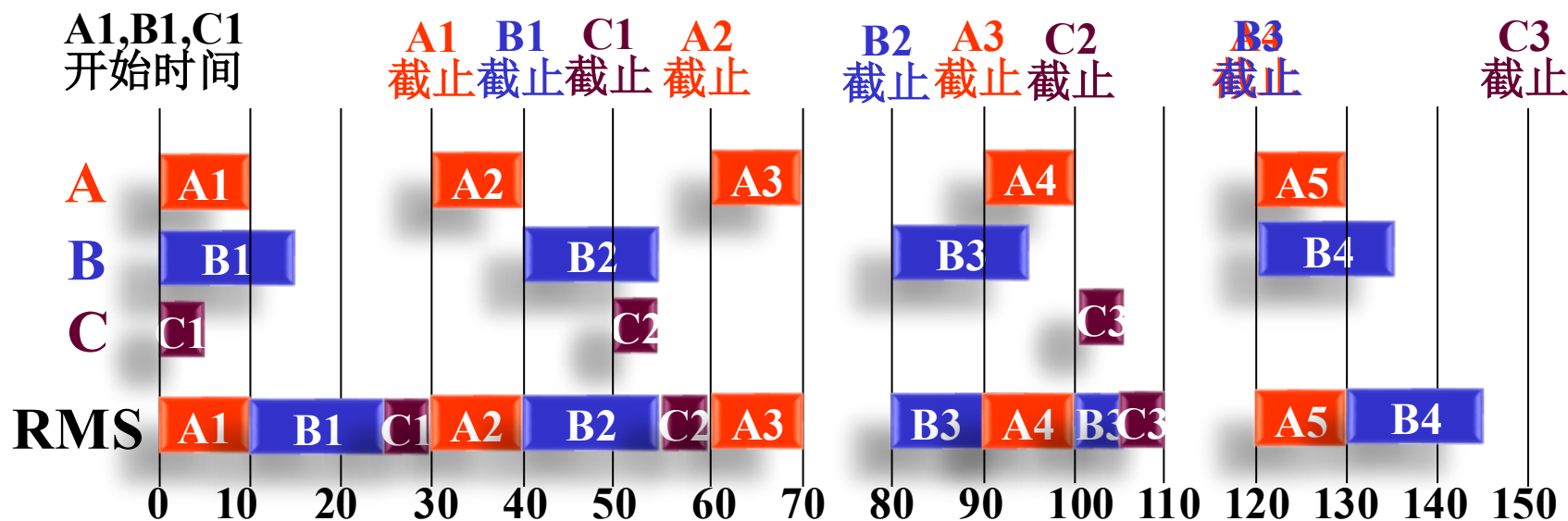


7.6 实时调度

7.6.7 静态优先级驱动的抢占调度

速率单调算法 (RMS-rate monotonic Scheduling)

- 3个进程A、B、C，A每30ms必须运行一次(10ms/次)，B每40ms一次(15ms/次)，C每50ms一次(5ms/次)
- 优先级：A>B>C





7.6 实时调度

7.6.8 动态分析调度算法 (Dynamic planning-based scheduling)

- **原理：** 在一个任务已到达但未执行时，试图创建一个包含前面被调度任务和新到达任务的调度。如果新到达任务可以按这种方式调度：满足它的最后期限，但是当前被调度的任务也不会错过它的最后期限，则修订这个调度以适应新任务。
- **特点：** 动态分析，而不是在开始运行前离线地确定。可用于确定何时分派任务。



7.6 实时调度

7.6.9 无保障动态调度算法 (动态最大努力调度—— Dynamic best effort scheduling)

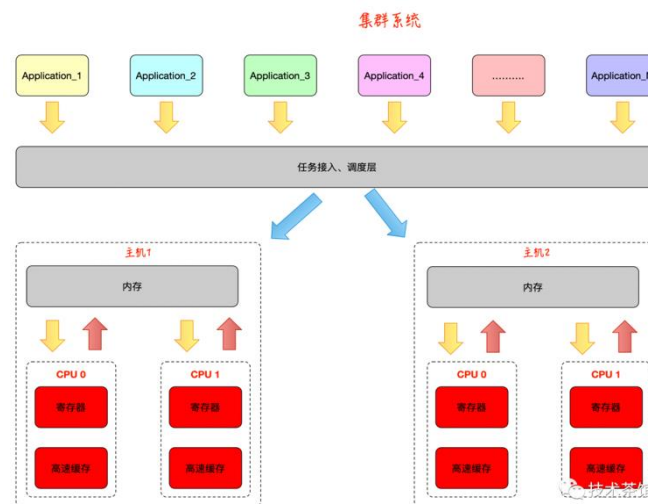
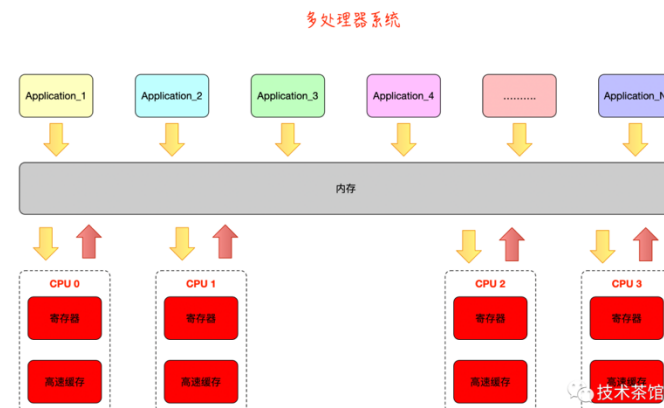
- **原理：**在任务下达时，根据任务的特性给它分配一个优先级。并通常使用某种形式的限期调度。开始执行，在时限到达时未完成任务被取消。
- **特点：**
 - 用于非周期性任务的实时系统；
 - 易于实现；
 - 试图满足所有的最后期限。



7.7 多处理机调度

7.7.1 与单处理机调度的区别

- **多处理器系统**是包含两个或更多处理器的计算机系统。在结构和管理上也变得更为复杂。分为**松散耦合多处理器系统**（或称**集群系统**）、**紧密耦合多处理器系统**。
- 多处理机调度MPS, **MultiProcessor Scheduling**
 - 注重**整体运行效率**（而不是个别处理机的利用率）；
 - **更多样的调度算法**
 - 多处理机访问OS数据结构时的**互斥**（对于共享内存系统）；
 - **调度单位**广泛采用线程。
 - **负载均衡**和**进程迁移**





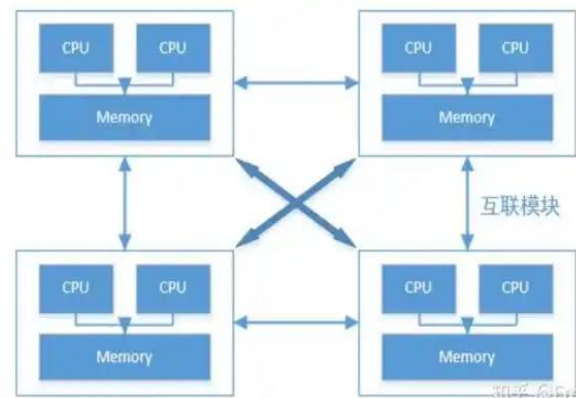
7.7 多处理机调度

7.7.1 与单处理机调度的区别

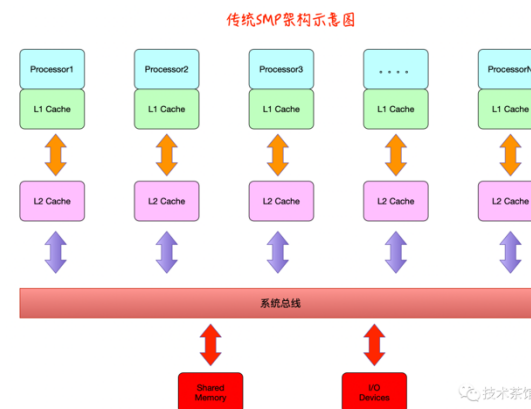
● 根据各处理器是否**相同**，MPS可分为：

➤ **对称多处理器系统SMPS**（Symmetric MultiProcessor System）：系统所包含的各处理器单元，在功能和结构上都是相同的，所有处理器的地位都是相同的，所有的资源特别是存储器、中断及I/O空间，都具有相同的可访问性，消除了结构上的障碍。

➤ **非对称多处理器系统ASMP**（Asymmetric Multi-Processor）：是将若干个处理器挂接到总线上，在各处理器之间形成简单的主从设备关系。ASMP不允许所有处理器访问所有系统资源，使系统性能受到限制。其中包含一个**主处理器**，多个**从处理器**。



NUMA架构图示 <http://blog.csdn.net/51CTO博客>



传统SMP架构示意图



7.7 多处理机调度

7.7.2 对称式多处理系统 (SMP)

- 按控制方式，SMP调度算法可分为集中控制和分散控制。静态和动态调度都是集中控制，而自调度是分散控制。
- 集中控制
 - 静态分配(static assignment): 每个CPU设立一个就绪队列，进程从开始执行到完成，都在同一个CPU上。类似现实生活中的什么？
 - 优点：调度算法开销小
 - 缺点：容易出现忙闲不均
 - 动态分配(dynamic assignment): 所有CPU采用一个公共就绪队列，队首进程每次分派到当前空闲的CPU上执行。类似现实生活中的什么？



7.7 多处理机调度

7.7.2 对称式多处理系统 (SMP)

按控制方式，SMP调度算法可分为**集中控制**和**分散控制**。静态和动态调度都是集中控制，而自调度是分散控制。

● 分散控制

➤ **自调度(self-scheduling)**: 各个CPU采用一个**公共就绪队列**，每个处理机都可以从队列中选择适当进程来执行。需要对就绪队列的数据结构进行**互斥访问控制**。它是最常用的算法，实现时易于移植采用单处理机的调度技术。

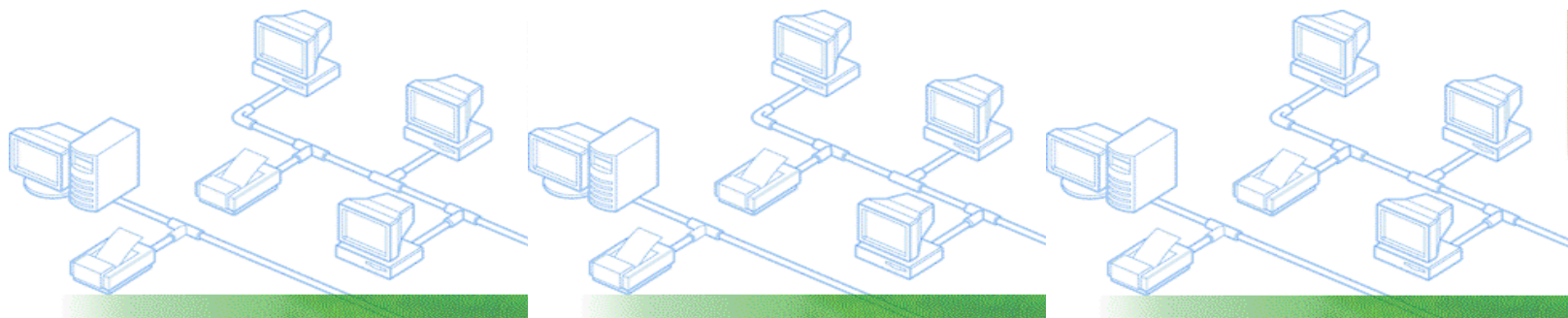
● **变型**: Mach OS中局部和全局就绪队列相结合，其中局部就绪队列中的线程优先调度。



7.7 多处理机调度

7.7.3 非对称式多处理系统(ASMP)的处理机调度

- 主-从处理机系统，由主处理机管理一个公共就绪队列，并分派进程给从处理机执行
- 各个处理机有固定分工，如执行OS的系统功能，I/O处理，应用程序





7.7 多处理机调度

7.7.4 成组调度 (Gang scheduling)

- 将一个进程中的一组线程，每次分派时同时分到一组处理机上执行，在剥夺处理机时也同时对这一组线程进行

如何为应用程序
分配处理器时间？

- 优点

- 通常这样的一组线程在应用逻辑上相互合作，成组调度提高了这些线程的执行并行度，有利于减少阻塞和加快推进速度，最终提高系统吞吐量
- 每次调度可以完成多个线程的分派，在系统内线程总数相同时能够减少调度次数，减少调度算法的开销



7.7 多处理机调度

7.7.4 成组调度 (gang scheduling)

● 处理器时间分配

- 面向所有**应用程序**平均分配处理器时间：系统中 N 个处理器和 M 个应用程序，每个应用程序最多有 $1/M$ 时间去占有 N 个处理器

应用程序A 应用程序B

处理器1	线程1	线程1
处理器2	线程2	空闲
处理器3	线程3	空闲
处理器4	线程4	空闲

1/2

1/2

浪费37.5%



7.7 多处理机调度

7.7.4 成组调度 (gang scheduling)

● 处理器时间分配

- 面向所有线程平均分配处理器时间：系统中 M 个应用程序，共 N 个线程，每个线程最多有 $1/N$ 时间去占有 N 个处理器

应用程序A 应用程序B

处理器1	线程1	线程1
处理器2	线程2	空闲
处理器3	线程3	空闲
处理器4	线程4	空闲

4/5

1/5

浪费15%



7.7 多处理机调度

7.7.5 专用处理机调度 (dedicated processor assignment)

- 为进程中的每个线程都固定分配一个CPU，直到该线程执行完成；
- **缺点：** 线程阻塞时，造成CPU的闲置；
- **优点：** 线程执行时不需切换，相应的开销可以大大减小，推进速度更快；
- **适用场合：** CPU数量众多的高度并行系统，单个CPU利用率已不太重要。



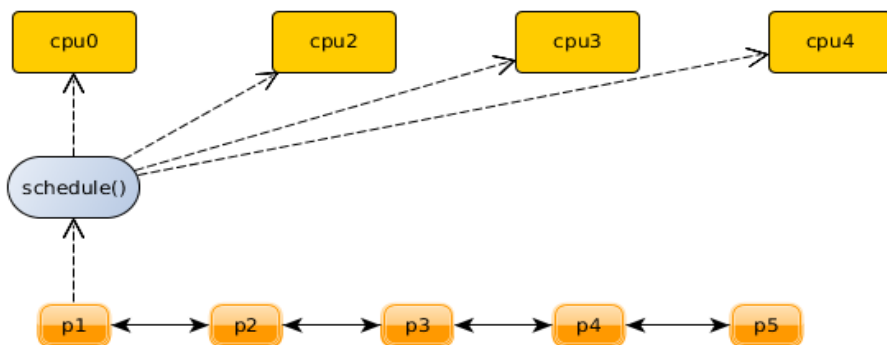
7.8 Linux (openEuler) 的处理机调度

- ❖ Linux既支持普通的**分时进程**，也支持**实时进程**
- ❖ Linux中的调度是**多种调度策略和调度算法的混合**。
 - ❖ Linux-4.6内核实现了6种调度策略
 - 普通的**非实时进程**的调度策略：
 - SCHED_NORMAL、SCHED_BATCH
 - **实时进程**调度策略：
 - SCHED_FIFO、SCHED_RR、SCHED_DEADLINE
 - 系统空闲时调用idle进程： SCHED_IDLE
 - Linux进程调度算法演变
 - Linux 2.4的 **$O(n)$** 调度算法：一个调度队列，遍历找下一个
 - Linux 2.6.17的 **$O(1)$** 调度算法（2.6.23之前）：两个140个优先级队列数组（active、expired）
 - Linux 2.6.26的调度算法： CFS，虚拟运行时间、红黑树组织就绪队列



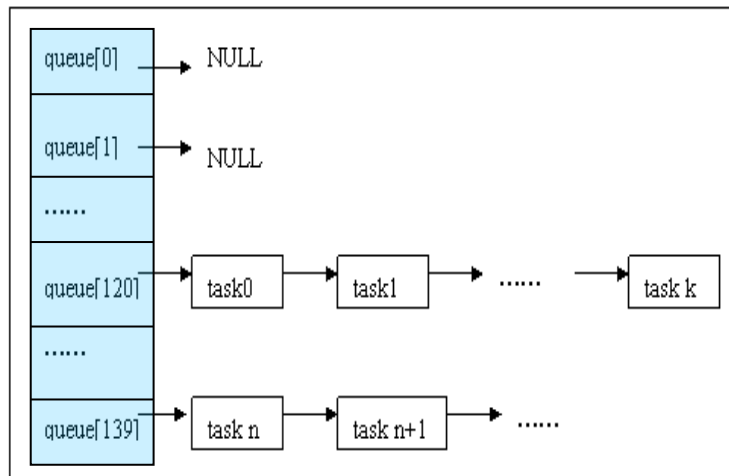
7.8 Linux (openEuler) 的处理机调度

2.4的 $O(n)$ 调度

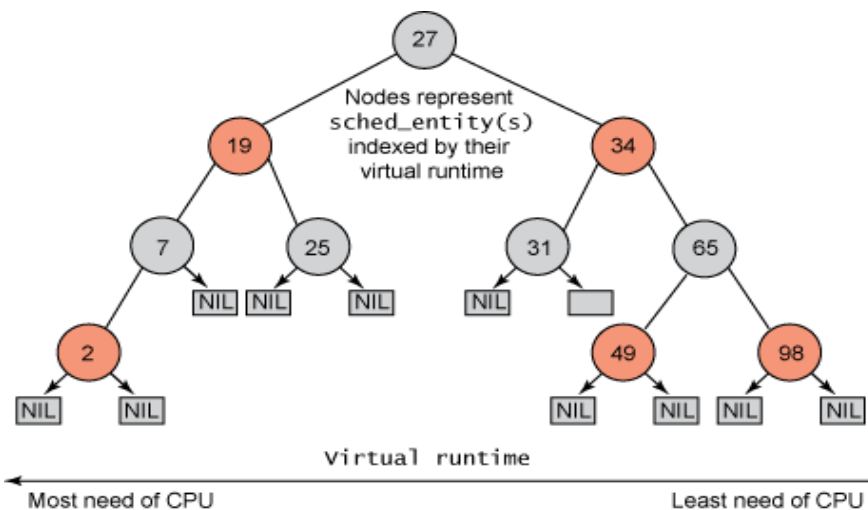


2.6.17-2.6.23的 $O(1)$

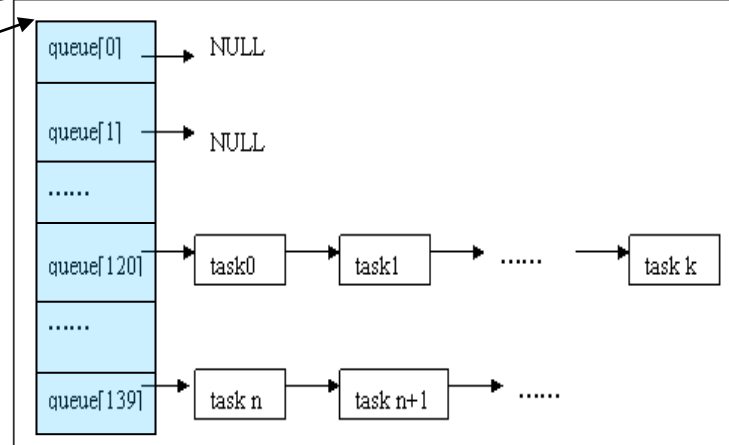
active



2.6.23以后: CFS调度



expired





7.8 Linux (openEuler) 的处理机调度

CFS调度器简介

- CFS使用红黑树组织就绪队列
- CFS彻底以时间和系统负载为参照
- 对于普通进程, 不再区分IO消耗型还是CPU消耗型
- 不再使用时间片的概念,
- 使用虚拟运行时间作为调度的依据: 与运行时间成正比、与进程的权重成反比, 权重与优先级成相同的趋势
- 引入了模块化调度: 调度实体和调度类
- CFS引入了完全公平调度、组调度等特性
- 对SMP负载均衡进行了重构
- 实现代码在: **kernel/sched_fair.c**



7.8 Linux (openEuler) 的处理机调度

CFS调度类

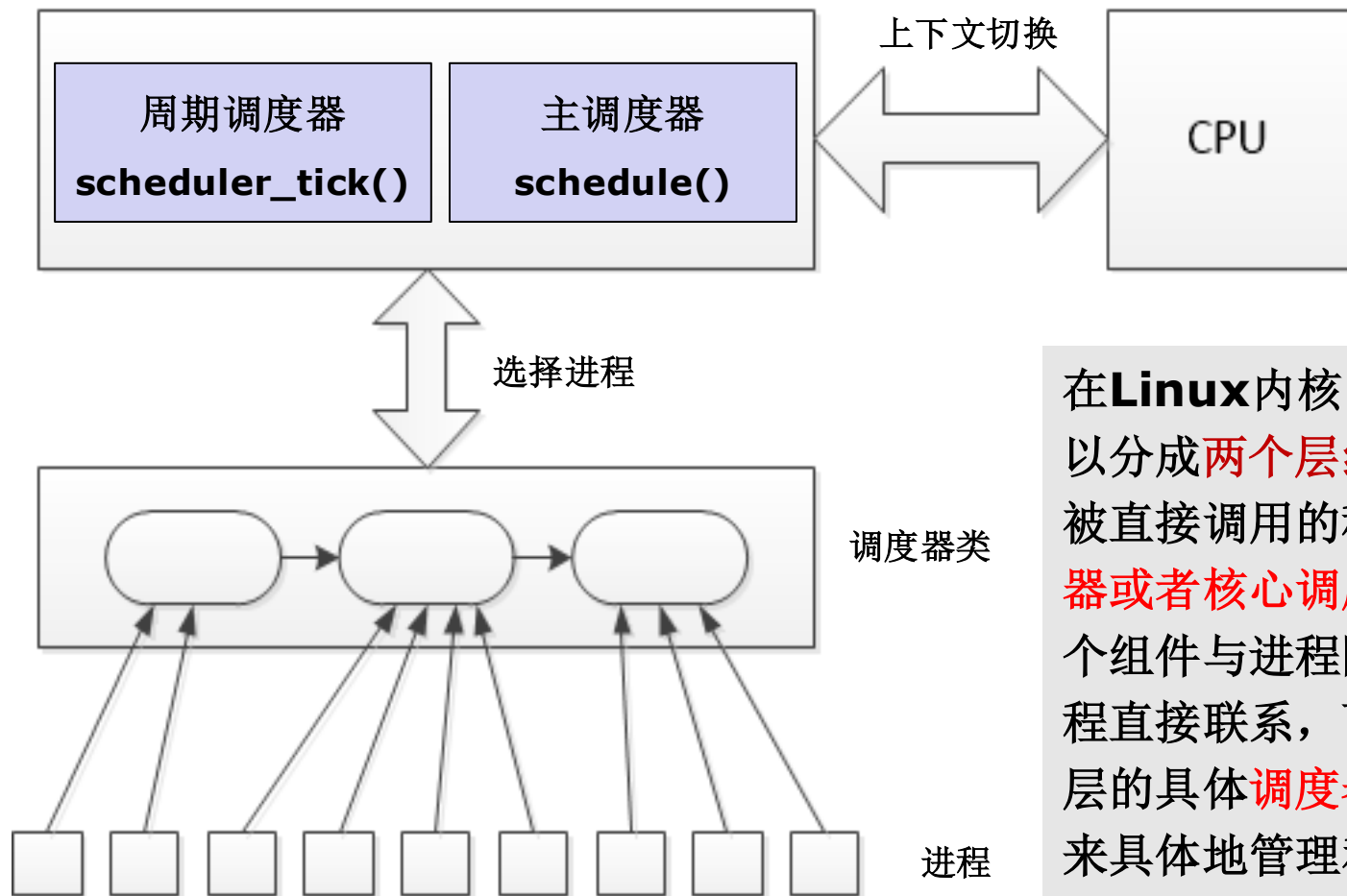
```
static const struct sched_class fair_sched_class = {
    .next                = &idle_sched_class,
    .enqueue_task        = enqueue_task_fair,    //将进程放入红黑树
    .dequeue_task        = dequeue_task_fair,
    .yield_task          = yield_task_fair,
    .yield_to_task        = yield_to_task_fair,
    .check_preempt_curr  = check_preempt_wakeup,
    .pick_next_task      = pick_next_task_fair, //选择下一个被调度运行的进程
    .put_prev_task       = put_prev_task_fair,
#ifdef CONFIG_SMP
    .....
#endif
    .set_curr_task       = set_curr_task_fair,
    .task_tick           = task_tick_fair,
    .task_fork           = task_fork_fair,
    .prio_changed        = prio_changed_fair,
    .switched_from       = switched_from_fair,
    .switched_to         = switched_to_fair,
    .get_rr_interval     = get_rr_interval_fair,
#ifdef CONFIG_FAIR_GROUP_SCHED
    .task_move_group     = task_move_group_fair,
#endif
};
```

kernel/sched_fair.c



7.8 Linux (openEuler) 的处理机调度

❖ Linux高版本的调度器框架

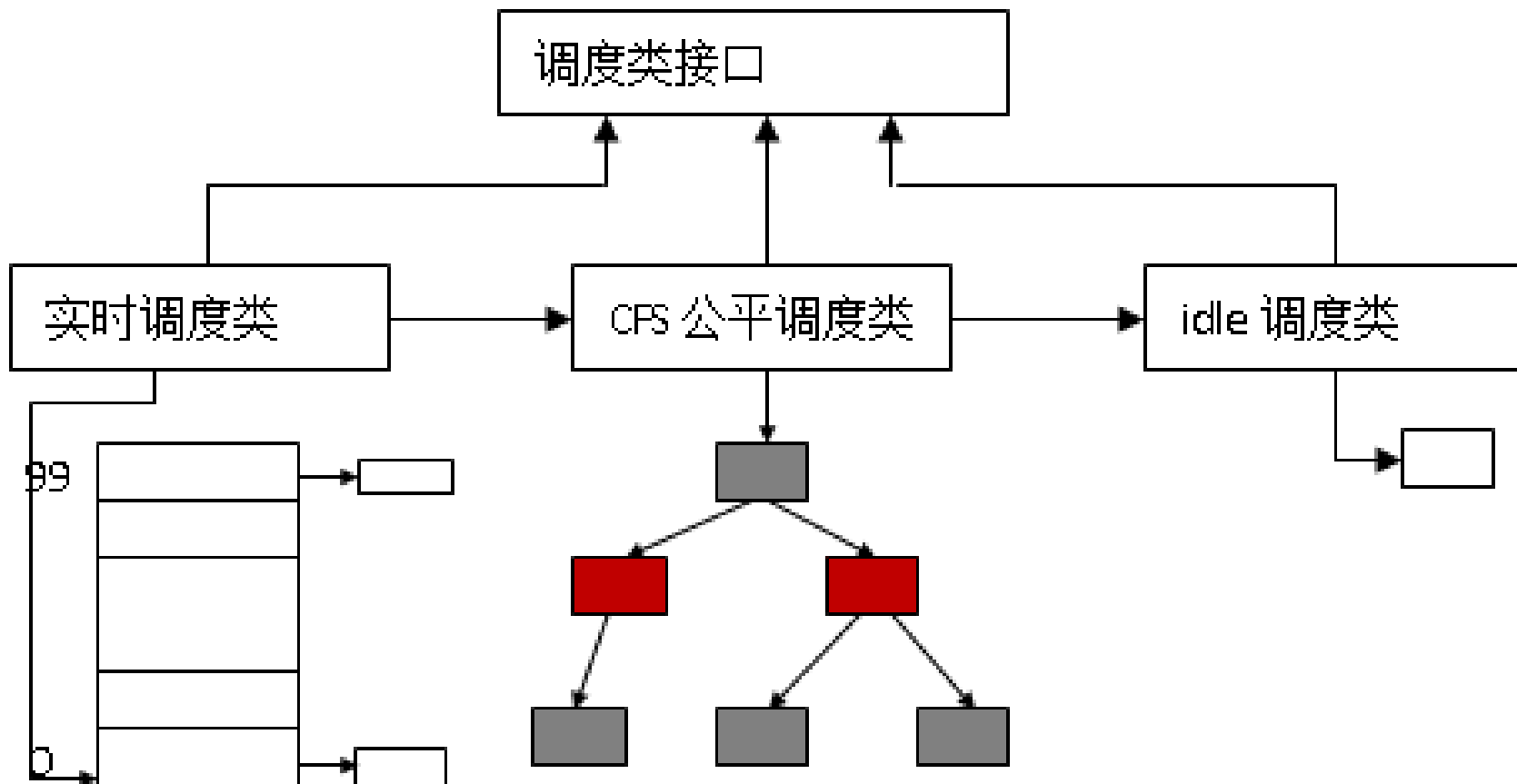


在Linux内核中，调度器可以分成两个层级，在内核中被直接调用的称为通用调度器或者核心调度器，作为一个组件与进程隔离，不与进程直接联系，而是通过第二层的具体调度器类（接口）来具体地管理和调度进程。



7.8 Linux (openEuler) 的处理机调度

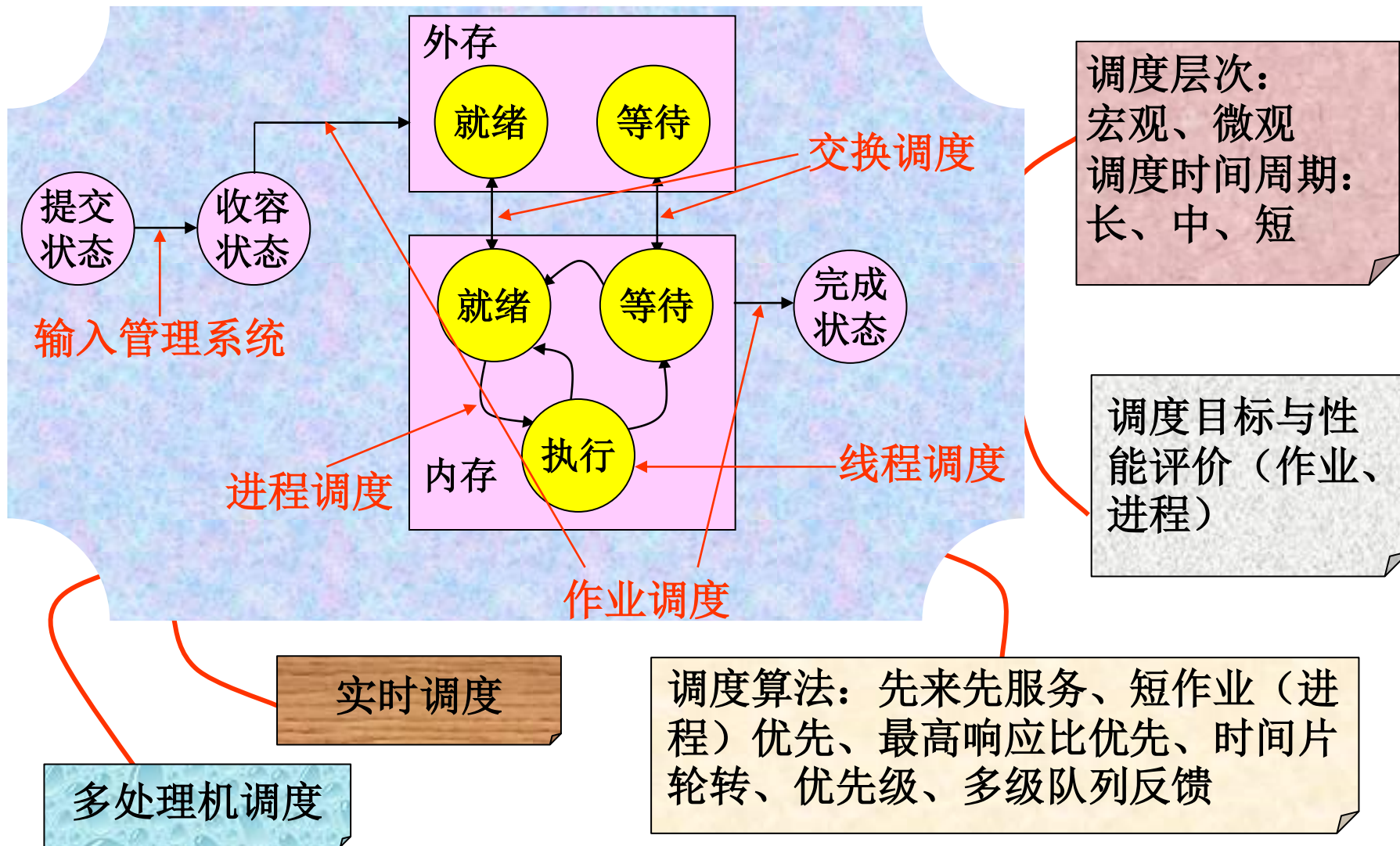
❖ Linux 的调度类接口



- 每个处理器定义一个 **rq** 结构，其中包含三种就绪队列和一些管理信息



第七章小结





继续学习第8章： 存储管理

